Data Structures & Algorithms Ultimate Notes

AVL Trees · B-Trees & B+ Trees · Hashing · Bloom Filters
Heaps · Graphs · Minimum Spanning Trees

Compiled from the IIT Hyderabad lecture notes
(AVL, B-Tree/Hashing/Bloom, Hashing, Heaps, Graphs, MST)

How to read this document. Every topic has: intuition → definitions → a *simple example* (green box) → the *example from the lecture notes* (orange box) → pseudocode → C code → proofs → complexity (yellow box). All C listings were compiled and tested (random stress tests, AddressSanitizer), and every worked example from the notes was reproduced by the code. Sections marked *(supplement)* are not in the lecture slides but are needed to complete the topic.

Contents

1 AVL Trees **4**

1.1 Why balance a binary search tree? . . . 4
1.2 The height of an AVL tree is $\mathcal{O}(\log n)$ . . . 4
1.3 Rotations . . . 5
1.4 Insertion . . . 6
1.5 Deletion . . . 9
1.6 Complexity summary . . . 10

2 B-Trees and B+ Trees **12**

2.1 Motivation: data that lives on disk . . . 12
2.2 B-trees . . . 12
2.2.1 Height of a B-tree (proof) . . . 13
2.2.2 Node definition in C . . . 13
2.2.3 Search . . . 14
2.2.4 Insertion . . . 14
2.2.5 Deletion . . . 16
2.3 B+ trees . . . 18
2.3.1 Node, search, range query . . . 19
2.3.2 Insertion (supplement) . . . 20
2.3.3 Deletion (supplement) . . . 21
2.4 B-tree vs. B+ tree . . . 22

3 Hashing **24**

3.1 Can we search in constant time? . . . 24
3.2 The dictionary problem and direct addressing . . . 24
3.3 Chaining . . . 25
3.3.1 Load factor and simple uniform hashing . . . 26
3.3.2 Search time with chaining . . . 27
3.3.3 Chaining in C (with rehashing) . . . 27
3.4 Open addressing . . . 28
3.4.1 Three probe sequences . . . 28
3.4.2 Clustering . . . 29
3.4.3 Deletion . . . 29
3.4.4 Open addressing in C . . . 30
3.5 Hash functions . . . 31

3.6 Universal hashing . . . 31
3.6.1 Family 1: $h_{a,b}(k) = ((ak+b) \bmod p) \bmod m$ . . . 32
3.6.2 Family 2: dot product modulo a prime . . . 32
3.7 Perfect hashing (static sets) . . . 33
3.8 Solved quiz questions . . . 33

4 Bloom Filters **34**
4.1 The membership problem . . . 34
4.2 Structure: m bits and k hash functions . . . 34
4.3 One-sided error . . . 36
4.4 Analysis . . . 36
4.5 Operations a Bloom filter cannot do (and variants) . . . 37
4.6 Applications . . . 37

5 Heaps and Priority Queues **39**
5.1 Priority queues . . . 39
5.2 Definition: two properties . . . 39
5.2.1 Array representation (1-indexed) . . . 39
5.2.2 Height and size . . . 40
5.2.3 Consequences of the max-heap property . . . 40
5.3 Insert (sift up) . . . 40
5.4 Delete-Max and Max-Heapify (sift down) . . . 41
5.5 Heap sort . . . 42
5.6 Building a heap . . . 42
5.7 More priority-queue operations . . . 43
5.8 C code . . . 43

6 Graphs **45**
6.1 Basic definitions . . . 45
6.2 Representations . . . 45
6.3 Exploring a graph: depth-first search . . . 46
6.3.1 Connected components . . . 47
6.4 Pre- and post-visit numbers . . . 47
6.5 DFS in directed graphs . . . 48
6.6 DAGs and topological sort . . . 49
6.7 Strongly connected components . . . 50
6.8 Breadth-first search (supplement) . . . 51
6.9 Planar graphs and Euler's formula . . . 52

7 Minimum Spanning Trees **53**

7.1 Spanning trees and the MST problem . . . 53
7.2 The generic algorithm and the cut property . . . 53
7.3 Kruskal's algorithm . . . 54
7.3.1 Correctness . . . 55
7.3.2 Running time . . . 56
7.4 Disjoint-set (union-find) data structure . . . 56
7.4.1 Linked lists with weighted union . . . 56
7.4.2 Disjoint-set forests (supplement) . . . 57
7.5 Prim's algorithm (supplement) . . . 58

8 Quick Reference **60**
8.1 Complexity cheat sheet . . . 60
8.2 Key formulas . . . 60
8.3 When to use what . . . 60

Chapter 1

AVL Trees

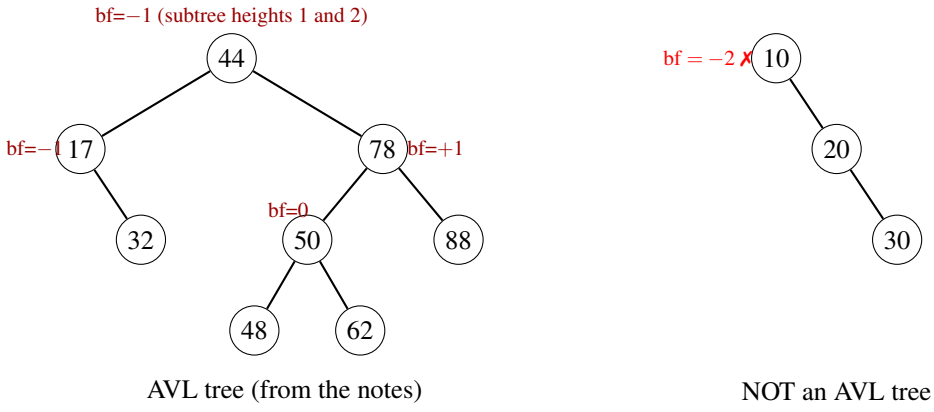
1.1 Why balance a binary search tree?

A binary search tree (BST) stores keys so that for every node, all keys in the left subtree are smaller and all keys in the right subtree are larger. Search, insert and delete walk one root-to-leaf path, so they cost $\mathcal{O}(\text{height})$. But the height of a BST depends on the insertion order: inserting $10, 20, 30, 40, \ldots$ in sorted order builds a *chain* of height $n-1$, and every operation becomes $\Theta(n)$.

An **AVL tree** (Adelson-Velsky and Landis, 1962) is a BST that *repairs itself* with small local changes (rotations) so that its height always stays $\mathcal{O}(\log n)$.

Definition (AVL property)

A binary search tree is an **AVL tree** if it satisfies all BST properties and, in addition, *for every node the heights of its left and right subtrees differ by at most 1.*
Height convention: an empty tree has height -1, a single node has height 0. The **balance factor** of a node v is $\text{bf}(v) = \text{height}(v.\text{left}) - \text{height}(v.\text{right}) \in \{-1, 0, +1\}$ in a valid AVL tree.



AVL tree (from the notes) NOT an AVL tree

Figure 1.1: Left: every node has subtree heights differing by ≤ 1. Right: node 10 has an empty left subtree (height -1) and a right subtree of height 1.

1.2 The height of an AVL tree is $\mathcal{O}(\log n)$

Claim

The height of an AVL tree with n nodes is $\mathcal{O}(\log n)$. More precisely $h \leq 2\log_2 n$ and $h < 1.4405\log_2(n+2)$.

Idea. Find the *fewest* nodes $N(h)$ an AVL tree of height h can have. If $N(h)$ grows exponentially in h, then h is logarithmic in n. The thinnest AVL tree of height h has a root, one subtree of height $h-1$ and one of height $h-2$, both themselves thinnest:

$$N(0) = 1, \qquad N(1) = 2, \qquad N(h) = 1 + N(h-1) + N(h-2) \ (h \geq 2).$$

So $N = 1, 2, 4, 7, 12, 20, 33, \ldots$

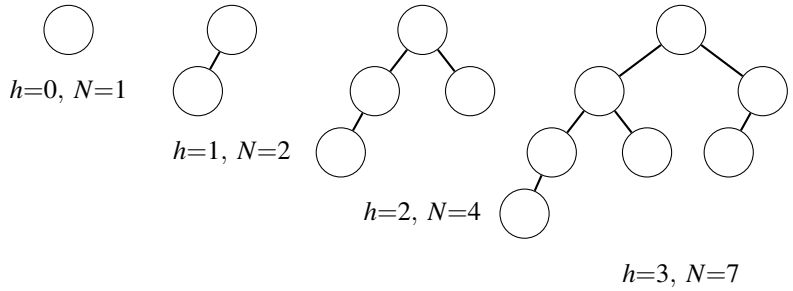


Figure 1.2: Thinnest AVL trees: T_h has a root with T_{h-1} and T_{h-2} as subtrees.

Proof 1 (simple): $h \le 2\log_2 n$

Clearly $N(h-1) \ge N(h-2)$, so $N(h) = 1 + N(h-1) + N(h-2) > 2N(h-2)$. Unrolling, $N(h) > 2^{\lfloor h/2 \rfloor}$, more precisely by induction $N(h) \ge 2^{h/2}$ (true for $h = 0, 1$; and $N(h) \ge 2N(h-2) \ge 2 \cdot 2^{(h-2)/2} = 2^{h/2}$). Hence $n \ge N(h) \ge 2^{h/2}$, i.e. $h \le 2\log_2 n = \mathcal{O}(\log n)$. □

Proof 2 (tight, golden ratio)

Let $M(h) = N(h) + 1$. Then $M(h) = M(h-1) + M(h-2)$ with $M(0) = 2$, $M(1) = 3$: this is the Fibonacci recurrence, so $M(h) = F_{h+3}$ ($F_1 = F_2 = 1$). Try the solution $M(h) = c^h$: $c^h = c^{h-1} + c^{h-2} \Rightarrow c^2 - c - 1 = 0 \Rightarrow c = \frac{1 \pm \sqrt{5}}{2}$. The positive root is the golden ratio $\varphi \approx 1.618$. Since $F_k \ge \varphi^{k-2}$, we get $n + 1 \ge N(h) + 1 = F_{h+3} \ge \varphi^{h+1}$, so

$$h \le \log_\varphi(n+1) - 1 \approx 1.44 \log_2(n+1) - 1.$$

So an AVL tree is at most 44% taller than a perfectly balanced tree. □

Proof 3 (the "least-depth leaf" argument from the notes)

Let x be a leaf of *least* depth k. Then the tree is a full binary tree down to depth $k-1$ (no leaf above depth k, so every node above has two children), hence the $2^k - 1$ nodes above depth k plus x give $n \ge 2^k$, i.e. $k \le \log_2 n$. Now bound the height from x upward: x has height 0. If a node on the path has a child on the path of height $\le 2(d-1)$, its other child has height $\le 2(d-1) + 1$ (AVL), so the node's height is $\le 2d$. The root is k levels above x, so $h \le 2k \le 2\log_2 n$. □

1.3 Rotations

A **rotation** reorganises a BST locally while keeping the in-order sequence (so it *still satisfies the BST property*) and changes only a constant number (≈ 6) of pointers.

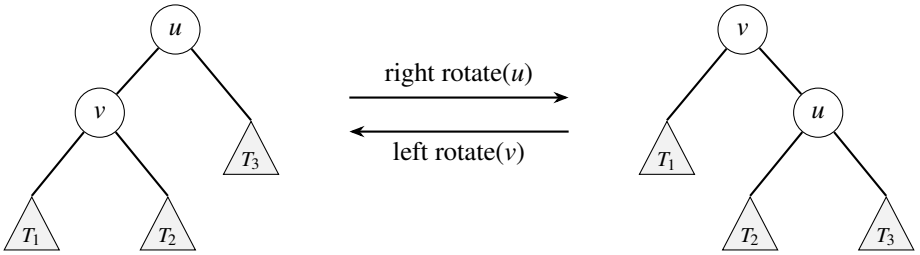
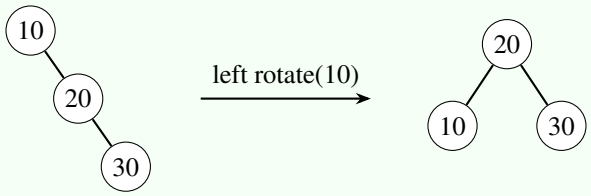


Figure 1.3: Right rotation about u lifts its left child v; left rotation about v undoes it. In-order: $T_1\, v\, T_2\, u\, T_3$ in both trees.

Simple example: rotating a 3-chain

Insert $10, 20, 30$ into an empty AVL tree. After 30, node 10 has balance -2 (a right-right chain). One *left rotation about 10* makes 20 the root with children 10 and 30.



C code: rotations

```c
/* Right rotate about u: v = u->left becomes the root of this subtree. */
static Node *rotate_right(Node *u) {
    Node *v = u->left;
    u->left = v->right;          /* T3 moves across              */
    v->right = u;
    update(u); update(v);        /* u first: it is now the child  */
    return v;
}

/* Left rotate about v: u = v->right becomes the root of this subtree. */
static Node *rotate_left(Node *v) {
    Node *u = v->right;
    v->right = u->left;
    u->left = v;
    update(v); update(u);
    return u;
}
```

1.4 Insertion

Algorithm. (1) Insert as in a normal BST (new leaf). (2) Walk back up toward the root, recomputing heights. (3) At the *first* node z whose balance becomes ± 2, rotate; the tree is then valid again.

Let z be the lowest unbalanced node, y its child on the path to the new key and x the grandchild on that path. Four cases:

Case	Shape	Fix	Rotations
LL	y left of z, x left of y (straight line)	right rotate z	1
RR	y right of z, x right of y (straight line)	left rotate z	1
LR	y left of z, x right of y (zig-zag)	left rotate y, then right rotate z	2 (double)
RL	y right of z, x left of y (zig-zag)	right rotate y, then left rotate z	2 (double)

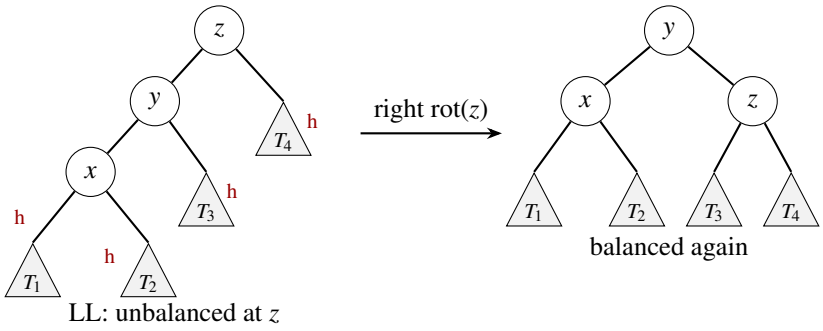


Figure 1.4: Case LL (the others are mirror images). Before insertion $T_1 \dots T_4$ all have height about h; the insertion into T_1 made it $h+1$.

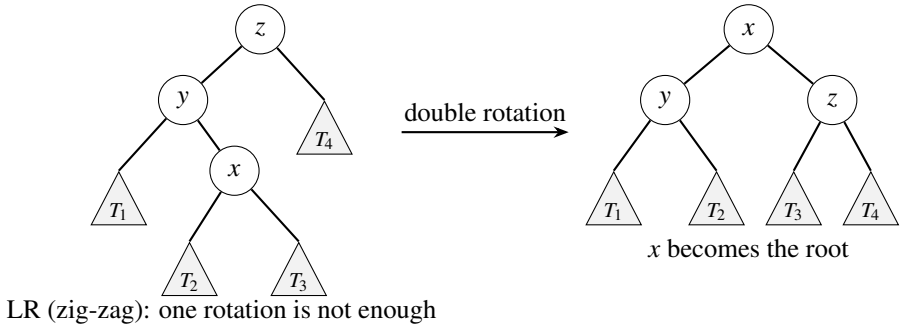


Figure 1.5: Case LR: left rotate y (turns it into LL), then right rotate z. Net effect: x moves to the top.

Why one fix suffices (proof)

After the first rotation the whole tree is an AVL tree again

Let z be the lowest unbalanced node and let its subtree have height $h+2$ *before* the insertion. The insertion raised the height of the child subtree containing the new key; since z had balance in $\{0,\pm1\}$ before and is ±2 now, it must have been *already* leaning to that side: the heavy child y had height $h+1$ and the other child T_4 height h; after insertion y's subtree has height $h+2$ and z's has height $h+3$. Because y is balanced and its height *grew*, one of its children (x's side) has height $h+1$ and the other has height exactly h (otherwise y's height would not have changed).

Straight case (LL). After right-rotating z: z has children of heights h and h, so height $h+1$ and balance 0; y has children of heights $h+1$ (x) and $h+1$ (z), so balance 0 and height $h+2$.

Zig-zag case (LR). x has height $h+1$ with children of heights h and h or h and $h-1$. After the double rotation x is root, y has children of heights h and $\{h,h-1\}$ (height $h+1$), z has children $\{h,h-1\}$ and h (height $h+1$); both are balanced and x is balanced with height $h+2$.

In every case the new subtree height is $h+2$, *the same as before the insertion*. So no ancestor of z sees any change, and no further rotation is needed. □

Pseudocode

Algorithm 1: AVL-Insert(t,k)

```
1 if t = NIL then return new node(k);
2 if k < t.key then t.left ← AVL-Insert(t.left, k);
3 else if k > t.key then t.right ← AVL-Insert(t.right, k);
4 else return t;
5 return Rebalance(t);
```

Algorithm 2: Rebalance(z)

update $z.height \leftarrow 1 + \max(height(z.left), height(z.right))$;
$b \leftarrow height(z.left) - height(z.right)$;
if $b > 1$ **then**
    if $balance(z.left) < 0$ **then** $z.left \leftarrow$ RotateLeft($z.left$);
    ▷ LR case
    return RotateRight(z)
if $b < -1$ **then**
    if $balance(z.right) > 0$ **then** $z.right \leftarrow$ RotateRight($z.right$);
    ▷ RL case
    return RotateLeft(z)
return z;

C code: insert

```c
typedef struct Node {
    int key;
    int height;                 /* height of subtree rooted here (leaf = 0) */
    struct Node *left, *right;
} Node;

static int height(Node *n) { return n ? n->height : -1; }   /* empty tree: -1 */
static int max(int a, int b) { return a > b ? a : b; }
static void update(Node *n) { n->height = 1 + max(height(n->left), height(n->right)); }
static int balance(Node *n) { return n ? height(n->left) - height(n->right) : 0; }

static Node *new_node(int key) {
    Node *n = malloc(sizeof(Node));
    n->key = key; n->height = 0; n->left = n->right = NULL;
    return n;
}
```

```c
/* Restore the AVL property at z (children are already AVL). Returns new root. */
static Node *rebalance(Node *z) {
    update(z);
    int b = balance(z);
    if (b > 1) {                                    /* left heavy              */
        if (balance(z->left) < 0)                   /* LR case: straighten     */
            z->left = rotate_left(z->left);
        return rotate_right(z);                     /* LL case                 */
    }
    if (b < -1) {                                   /* right heavy             */
        if (balance(z->right) > 0)                  /* RL case: straighten     */
            z->right = rotate_right(z->right);
        return rotate_left(z);                      /* RR case                 */
    }
    return z;
}
```

```c
Node *avl_insert(Node *t, int key) {
    if (!t) return new_node(key);
    if (key < t->key)      t->left  = avl_insert(t->left,  key);
    else if (key > t->key) t->right = avl_insert(t->right, key);
    else return t;                                  /* no duplicates           */
    return rebalance(t);                            /* fix on the way back up  */
}
```

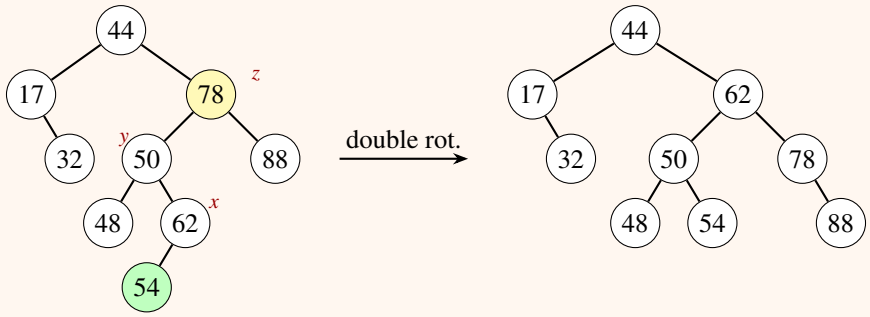
Simple example: all four cases

Insert in this order	Unbalanced node	Case / fix
$30, 20, 10$	30	LL: right rotate(30) $\Rightarrow$ root 20
$10, 20, 30$	10	RR: left rotate(10) $\Rightarrow$ root 20
$30, 10, 20$	30	LR: left rotate(10), right rotate(30) $\Rightarrow$ root 20
$10, 30, 20$	10	RL: right rotate(30), left rotate(10) $\Rightarrow$ root 20

The program prints `(10 20 30)` for both $10, 20, 30$ and $10, 30, 20$.

From your notes: insert 54 (a zig-zag at node 78)

The tree of the notes is $44\,(17\,(\cdot, 32),\ 78\,(50\,(48, 62),\ 88))$. Insert 54: $54 > 44 \rightarrow$ right; $54 < 78 \rightarrow$ left; $54 > 50 \rightarrow$ right to 62; $54 < 62 \rightarrow$ left child of 62. Walking back up: 62 ok, 50 ok (heights 0 and 1), **78 is unbalanced** (left subtree 50 has height 2, right subtree 88 has height 0). So $z = 78$, $y = 50$, $x = 62$: y is the left child of z but x is the *right* child of y $\Rightarrow$ zig-zag (LR), use a **double rotation**.



Step 1: left rotate about y=50 (lifts 62); step 2: right rotate about z=78 (lifts 62 again). Now 62 is the root of the subtree with children $50\,(48, 54)$ and $78\,(\cdot, 88)$, whose height is 2 again. The program prints `((. 17 32) 44 ((48 50 54) 62 (. 78 88)))`, exactly this tree. Note that finding the unbalanced node costs $\mathcal{O}(\log n)$ but the repair itself is $\mathcal{O}(1)$.

1.5 Deletion

Which node is physically removed? Search for the key. If the node has two children, replace its key by its in-order successor (minimum of the right subtree) and delete that successor instead. The physically deleted node is therefore a *leaf or a node with one child* (and if it has one child, AVL property forces that child to be a leaf).

Rebalancing. Let z be the first unbalanced ancestor of the removed node. Let y be the child of z with the **larger height**, and x the child of y with the larger height; *if both children of y have equal height, choose x on the same side as y* (so that x, y, z form a straight line and a single rotation works). Then apply the same four cases as for insertion.

Insertion vs deletion

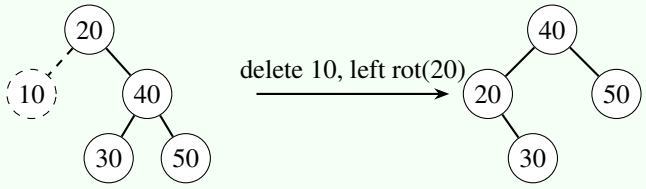
Insertion: one (single or double) rotation restores everything.
Deletion: after the rotation the height of the subtree may have *decreased by one*, so a parent of z can become unbalanced; we may have to repeat up to the root: $\mathcal{O}(\log n)$ rotations in the worst case. This is why the C code calls `rebalance` at every node on the way back.

Why the tie rule matters

Suppose z is unbalanced because one subtree dropped from height h to $h-1$ while the sibling y-subtree has height $h+1$. y's children have heights h and h or h and $h-1$. If we pick the tied zig-zag child x and do a double rotation we are fine, but if y's heavier side is the *outer* one the correct fix is a single rotation. With a tie, the single rotation gives y children of heights h and $h+1$ (balanced; subtree height $h+2$, i.e. the height did *not* drop). A double rotation on a tie, by contrast, would leave an imbalance. Hence: straight line whenever heights tie. □

Simple example: delete needing a single rotation

Tree $20\,(10,\ 40\,(30,50))$. Delete 10: node 20 now has left height -1 and right height 1, balance -2. So $z=20$, $y=40$ (taller). Both children of y have height 0 (tie) $\Rightarrow$ choose $x=50$ (same side as y): straight line RR $\Rightarrow$ single left rotation about 20.



Output of the program: `((.  20 30) 40 50).`

C code: delete and search

```c
static Node *min_node(Node *t) { while (t->left) t = t->left; return t; }

Node *avl_delete(Node *t, int key) {
    if (!t) return NULL;
    if (key < t->key)      t->left  = avl_delete(t->left,  key);
    else if (key > t->key) t->right = avl_delete(t->right, key);
    else {
        if (!t->left || !t->right) {               /* leaf or one child       */
            Node *c = t->left ? t->left : t->right;
            free(t);
            return c;
        }
        Node *s = min_node(t->right);              /* in-order successor      */
        t->key = s->key;
        t->right = avl_delete(t->right, s->key);
    }
    return rebalance(t);                           /* may rotate at EVERY level */
}
```

```c
Node *avl_search(Node *t, int key) {
    while (t && t->key != key) t = key < t->key ? t->left : t->right;
    return t;
}
```

1.6 Complexity summary

Time / space complexity

Operation	Time	Notes
Search	$\mathcal{O}(\log n)$	height $\leq 1.44 \log_2(n+2)$
Insert	$\mathcal{O}(\log n)$	≤ 1 rotation (single or double)
Delete	$\mathcal{O}(\log n)$	up to $\mathcal{O}(\log n)$ rotations
Rotation	$\mathcal{O}(1)$	≈ 6 pointer updates
Space	$\Theta(n)$	one height (or balance) field per node

Chapter 2

B-Trees and B+ Trees

2.1 Motivation: data that lives on disk

AVL and red-black trees have height $\mathcal{O}(\log_2 n)$, perfect when everything is in RAM. But what if there are $n = 10^9$ records on disk?

Level	Access time
CPU registers / cache	$\sim$ 1 ns
Main memory (RAM)	$\sim$ 50–100 ns
Disk (SSD/HDD)	$\sim$ 0.1–10 ms

A disk access is about 10^5 times slower than a memory access, and the disk always reads a whole *page* (block) of 4–16 KB. So running time is dominated by DISK-READ/DISK-WRITE. **Key idea:** one node = one page; reading a node costs one disk access; therefore *minimise the number of nodes visited, i.e. the height*, by making nodes "fat" (hundreds of keys, hundreds of children).

2.2 B-trees

Definition (B-tree of minimum degree $t \geq 2$)

Each node x stores $x.n$ keys in sorted order $x.key_1 \leq \cdots \leq x.key_n$ and a flag $x.leaf$. An internal node has $x.n+1$ children $c_1, \ldots, c_{n+1}$ and the keys separate the child ranges: $k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq \cdots \leq x.key_n \leq k_{n+1}$ for any keys k_i in child c_i. **All leaves have the same depth** h. Node sizes are bounded:

Node	Keys	Children
Root (non-empty)	$1 \ldots 2t-1$	$2 \ldots 2t$ (if internal)
Other internal	$t-1 \ldots 2t-1$	$t \ldots 2t$
Leaf	$t-1 \ldots 2t-1$	0

A node with $2t-1$ keys is **full**. With $t = 2$ every internal node has 2, 3 or 4 children: the **2-3-4 tree**. In practice t is large (50–2000) so a node fills one disk page.

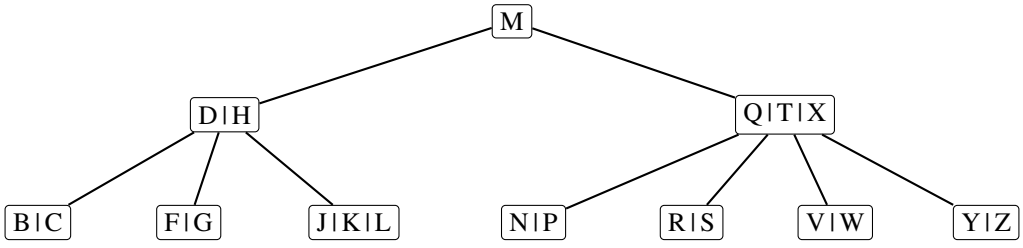
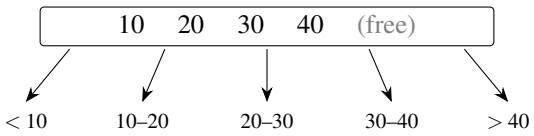


Figure 2.1: A B-tree with $t = 2$ (from the notes): a node with n keys has $n+1$ children, all leaves at depth 2.

A node with $t = 3$



$n = 4$ keys (max $2t - 1 = 5$) $\Rightarrow$ 5 child pointers; child c_i holds keys between key_{i-1} and key_i.

2.2.1 Height of a B-tree (proof)

Theorem (CLRS 18.1)

A B-tree with $n \geq 1$ keys, height h and minimum degree $t \geq 2$ satisfies $h \leq \log_t \frac{n+1}{2}$.

Proof

The root has ≥ 1 key and every other node has $\geq t-1$ keys. Count the *minimum* number of nodes per depth: 1 node at depth 0; 2 at depth 1; $2t$ at depth 2; $2t^2$ at depth 3; …; $2t^{h-1}$ at depth h (every non-root internal node has $\geq t$ children).

$$n \;\geq\; 1 + (t-1)\sum_{i=1}^{h} 2t^{i-1} \;=\; 1 + 2(t-1)\frac{t^h - 1}{t-1} \;=\; 2t^h - 1.$$

Hence $t^h \leq \frac{n+1}{2}$, i.e. $h \leq \log_t \frac{n+1}{2}$. □

n (keys)	Binary tree ($\log_2 n$)	B-tree $t = 100$	B-tree $t = 1000$
10^6	≈ 20	≤ 2	≤ 1
10^9	≈ 30	≤ 4	≤ 2
10^{12}	≈ 40	≤ 5	≤ 3

Larger t means fewer disk reads but more CPU work per node (scanning up to $2t-1$ keys; binary search inside the node reduces this to $\mathcal{O}(\log t)$).

2.2.2 Node definition in C

```c
#define T 3                          /* minimum degree (t >= 2)        */
#define MAX_KEYS (2 * T - 1)         /* a full node has 2t-1 keys      */

typedef struct BTreeNode {
    int n;                           /* number of keys stored          */
    int keys[MAX_KEYS];              /* keys in increasing order       */
    struct BTreeNode *c[2 * T];      /* child pointers (n+1 used)      */
    bool leaf;
} BTreeNode;

BTreeNode *allocate_node(bool leaf) {
    BTreeNode *x = malloc(sizeof(BTreeNode));
    x->n = 0; x->leaf = leaf;
    for (int i = 0; i < 2 * T; i++) x->c[i] = NULL;
    return x;
}
BTreeNode *btree_create(void) { return allocate_node(true); }
```

2.2.3 Search

Generalised BST search: in each node scan for the first key $\geq k$. If equal, done; otherwise descend into the matching child (one disk read). Visits $\leq h+1$ nodes.

Algorithm 3: B-Tree-Search(x,k)

$i \leftarrow 1$; **while** $i \leq x.n$ **and** $k > x.key_i$ **do** $i \leftarrow i+1$;
if $i \leq x.n$ **and** $k = x.key_i$ **then return** (x,i);
if $x.leaf$ **then return** NIL;
DISK-READ$(x.c_i)$; **return** B-Tree-Search$(x.c_i,k)$;

```c
/* ---------------- SEARCH ---------------- */
BTreeNode *btree_search(BTreeNode *x, int k, int *idx) {
    int i = 0;
    while (i < x->n && k > x->keys[i]) i++;          /* first key >= k */
    if (i < x->n && k == x->keys[i]) { *idx = i; return x; }
    if (x->leaf) return NULL;
    return btree_search(x->c[i], k, idx);            /* DISK-READ      */
}
```

2.2.4 Insertion

Challenge. A new key always goes into a leaf, but the leaf may be full. **Fix:** *split* a full node around its median key; the median moves up to the parent. If the parent is full, split it too. CLRS avoids "backing up" by splitting every full node met *on the way down*, so the recursion always enters a non-full node.

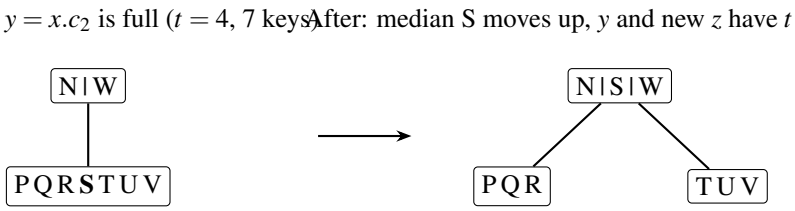
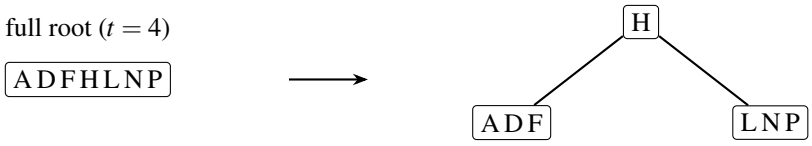


Figure 2.2: Splitting a full child: y keeps the lower $t-1$ keys, a new node z gets the upper $t-1$ keys, the median goes up. The tree does not get taller.

Splitting the root is the only way a B-tree grows in height: create an empty new root s, make the old root its child, split it. Height increases at the *top*, so all leaves stay at the same depth.



Algorithm 4: B-Tree-Insert(T,k)

$r \leftarrow T.root$;
if $r.n = 2t-1$ **then**
  allocate s; $T.root \leftarrow s$; $s.leaf \leftarrow$ false; $s.n \leftarrow 0$; $s.c_1 \leftarrow r$;
  B-Tree-Split-Child$(s,1)$; B-Tree-Insert-Nonfull(s,k) ▷ `height grows`
else
  B-Tree-Insert-Nonfull(r,k)

Algorithm 5: B-Tree-Insert-Nonfull(x,k)

if $x.leaf$ **then**
  shift keys larger than k one place right; insert k; $x.n \leftarrow x.n+1$
else
  find child index i with $key_{i-1} < k \le key_i$;
  if $x.c_i$ *is full* **then**
    B-Tree-Split-Child(x,i); **if** $k > x.key_i$ **then** $i \leftarrow i+1$;
  B-Tree-Insert-Nonfull$(x.c_i,k)$

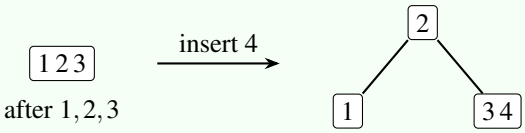
```c
/* ---------------- INSERT ---------------- */
void btree_split_child(BTreeNode *x, int i) {        /* x->c[i] is full */
    BTreeNode *y = x->c[i];
    BTreeNode *z = allocate_node(y->leaf);
    z->n = T - 1;
    for (int j = 0; j < T - 1; j++) z->keys[j] = y->keys[j + T];
    if (!y->leaf)
        for (int j = 0; j < T; j++) z->c[j] = y->c[j + T];
    y->n = T - 1;
    for (int j = x->n; j > i; j--) x->c[j + 1] = x->c[j];
    x->c[i + 1] = z;
    for (int j = x->n - 1; j >= i; j--) x->keys[j + 1] = x->keys[j];
    x->keys[i] = y->keys[T - 1];                     /* median moves up */
    x->n++;
}
```

```c
void btree_insert_nonfull(BTreeNode *x, int k) {
    int i = x->n - 1;
    if (x->leaf) {
        while (i >= 0 && k < x->keys[i]) { x->keys[i + 1] = x->keys[i]; i--; }
        x->keys[i + 1] = k;
        x->n++;
    } else {
        while (i >= 0 && k < x->keys[i]) i--;
        i++;
        if (x->c[i]->n == MAX_KEYS) {
            btree_split_child(x, i);
            if (k > x->keys[i]) i++;
        }
        btree_insert_nonfull(x->c[i], k);
    }
}

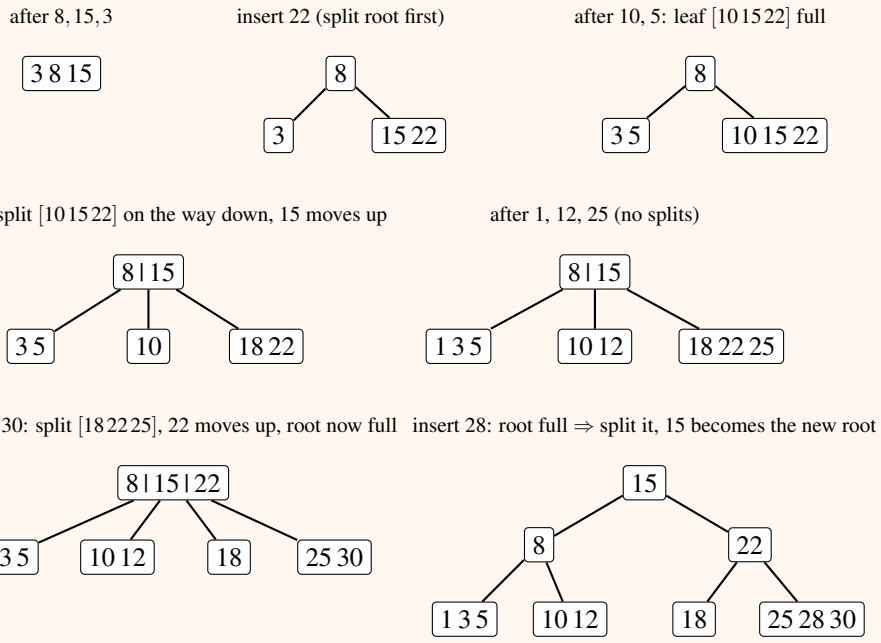
BTreeNode *btree_insert(BTreeNode *root, int k) {
    if (root->n == MAX_KEYS) {                       /* root full: split it */
        BTreeNode *s = allocate_node(false);
        s->c[0] = root;
        btree_split_child(s, 0);
        btree_insert_nonfull(s, k);
        return s;                                    /* height grows by one */
    }
    btree_insert_nonfull(root, k);
    return root;
}
```

Simple example: $t = 2$, insert $1,2,3,4$

Max 3 keys per node. After $1,2,3$ the root leaf is full: $[1\,2\,3]$. Insert 4: the root is full, so split it first: median 2 goes up into a new root, giving $[2]$ with leaves $[1]$ and $[3]$; then 4 goes into $[3] \rightarrow [3\,4]$.



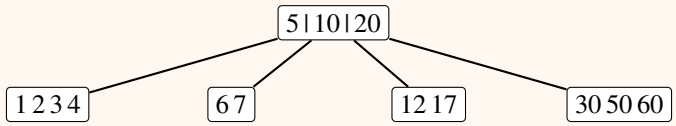
From your notes: $t = 2$: insert $8, 15, 3, 22, 10, 5, 18, 1, 12, 25, 30, 28$



This is the quiz question: after $8, 15, 3, 22$ the root contains only $[8]$ (answer A).

From your notes: sample C program, $t = 3$

Inserting $10, 20, 5, 6, 12, 30, 7, 17, 3, 4, 2, 50, 60, 1$ yields (root first, children indented): root [5 10 20] with children [1 2 3 4], [6 7], [12 17], [30 50 60]; and `search(17)` is found. This is exactly the output of our tested program.



2.2.5 Deletion

Strategy (from the notes). Keys can be deleted from *any* node. The danger is a node dropping below $t - 1$ keys. So **before descending into a child, make sure it has at least t keys**; then one key can always be removed without backing up. Only the root can become empty, and then the tree shrinks at the top (height decreases).

Case analysis (CLRS): let k be the key, x the current node (guaranteed $\geq t$ keys unless root).

1. k is in leaf x: just remove it.
2. k is in internal node x at position i; let y be the child before k and z the child after:
   - **2a.** y has $\geq t$ keys: replace k by its predecessor k' (largest key in y's subtree) and recursively delete k' from y.
   - **2b.** else z has $\geq t$ keys: replace k by its successor and recursively delete it from z.

2c. both have exactly $t-1$ keys: **merge** y, k, z into one node of $2t-1$ keys, free z, and recursively delete k from it.

3. k is not in x; it would be in subtree c_i. If c_i has only $t-1$ keys:

3a. a sibling has $\geq t$ keys: **borrow** (rotate) a key through the parent: the sibling's nearest key moves up, the separator moves down into c_i (with the sibling's nearest child pointer).

3b. both siblings have $t-1$ keys: **merge** c_i with one sibling and the separator from x.

then recurse into c_i.

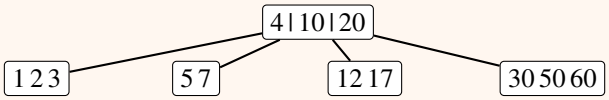
```c
/* ---------------- DELETE (CLRS style, one pass down) ---------------- */
static int find_key(BTreeNode *x, int k) {           /* first i with keys[i] >= k */
    int i = 0;
    while (i < x->n && x->keys[i] < k) i++;
    return i;
}
static void merge_children(BTreeNode *x, int i) {    /* merge c[i], keys[i], c[i+1]
    */
    BTreeNode *y = x->c[i], *z = x->c[i + 1];
    y->keys[T - 1] = x->keys[i];                     /* pull separator down */
    for (int j = 0; j < z->n; j++) y->keys[T + j] = z->keys[j];
    if (!y->leaf) for (int j = 0; j <= z->n; j++) y->c[T + j] = z->c[j];
    y->n += z->n + 1;
    for (int j = i + 1; j < x->n; j++) x->keys[j - 1] = x->keys[j];
    for (int j = i + 2; j <= x->n; j++) x->c[j - 1] = x->c[j];
    x->n--;
    free(z);
}
static void borrow_from_prev(BTreeNode *x, int i) {  /* rotate right through x->keys[
    i-1] */
    BTreeNode *c = x->c[i], *s = x->c[i - 1];
    for (int j = c->n - 1; j >= 0; j--) c->keys[j + 1] = c->keys[j];
    if (!c->leaf) for (int j = c->n; j >= 0; j--) c->c[j + 1] = c->c[j];
    c->keys[0] = x->keys[i - 1];
    if (!c->leaf) c->c[0] = s->c[s->n];
    x->keys[i - 1] = s->keys[s->n - 1];
    c->n++; s->n--;
}
static void borrow_from_next(BTreeNode *x, int i) {  /* rotate left through x->keys[i
    ] */
    BTreeNode *c = x->c[i], *s = x->c[i + 1];
    c->keys[c->n] = x->keys[i];
    if (!c->leaf) c->c[c->n + 1] = s->c[0];
    x->keys[i] = s->keys[0];
    for (int j = 1; j < s->n; j++) s->keys[j - 1] = s->keys[j];
    if (!s->leaf) for (int j = 1; j <= s->n; j++) s->c[j - 1] = s->c[j];
    c->n++; s->n--;
}
static void fill(BTreeNode *x, int i) {              /* make x->c[i] have >= T keys
    */
    if (i > 0 && x->c[i - 1]->n >= T)            borrow_from_prev(x, i);
    else if (i < x->n && x->c[i + 1]->n >= T)    borrow_from_next(x, i);
    else if (i < x->n)                           merge_children(x, i);
    else                                         merge_children(x, i - 1);
}
static int get_pred(BTreeNode *x, int i) { BTreeNode *c = x->c[i];   while (!c->leaf)
     c = c->c[c->n]; return c->keys[c->n - 1]; }
static int get_succ(BTreeNode *x, int i) { BTreeNode *c = x->c[i+1]; while (!c->leaf)
     c = c->c[0];    return c->keys[0]; }
```

```c
static void delete_from(BTreeNode *x, int k) {
    int i = find_key(x, k);
```

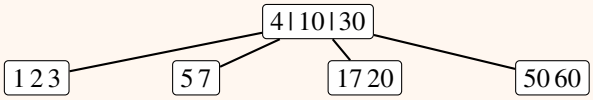
```
    if (i < x->n && x->keys[i] == k) {
        if (x->leaf) {                                  /* Case 1: key in a leaf    */
            for (int j = i + 1; j < x->n; j++) x->keys[j - 1] = x->keys[j];
            x->n--;
        } else if (x->c[i]->n >= T) {                   /* Case 2a: predecessor     */
            int p = get_pred(x, i); x->keys[i] = p; delete_from(x->c[i], p);
        } else if (x->c[i + 1]->n >= T) {               /* Case 2b: successor       */
            int s = get_succ(x, i); x->keys[i] = s; delete_from(x->c[i + 1], s);
        } else {                                        /* Case 2c: merge, recurse  */
            merge_children(x, i); delete_from(x->c[i], k);
        }
    } else {
        if (x->leaf) return;                            /* key not present          */
        bool last = (i == x->n);
        if (x->c[i]->n < T) fill(x, i);                 /* Case 3: ensure >= t keys */
        if (last && i > x->n) delete_from(x->c[i - 1], k);
        else                  delete_from(x->c[i], k);
    }
}
BTreeNode *btree_delete(BTreeNode *root, int k) {
    delete_from(root, k);
    if (root->n == 0 && !root->leaf) {                  /* root empty: tree shrinks */
        BTreeNode *old = root; root = root->c[0]; free(old);
    }
    return root;
}
```

From your notes: deletions on the $t = 3$ tree above (all cases)

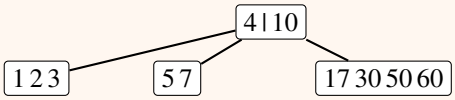
delete 6: leaf $[6\,7]$ has $t - 1 = 2$ keys; left sibling has $4 \Rightarrow$ borrow



delete 12: $[12\,17]$ minimal, left sibling minimal, right sibling $[30\,50\,60]$ has $3 \Rightarrow$ borrow from right (20 comes down, 30 goes up)



delete 20: $[17\,20]$ and both neighbours minimal $\Rightarrow$ merge $[17\,20] + 30 + [50\,60]$, then remove 20



delete 1: key in a leaf with 3 keys $\Rightarrow$ simply remove it: $[2\,3]$

Time / space complexity

Disk accesses (search / insert / delete)	$\mathcal{O}(h) = \mathcal{O}(\log_t n)$
CPU time	$\mathcal{O}(t \log_t n)$ with linear scan in nodes; $\mathcal{O}(\log n)$ with binary search
Space	$\Theta(n)$; nodes at least half full (except root)

2.3 B+ trees

Motivation: range queries. Databases ask "all students with marks between 60 and 80". In a B-tree keys are spread over all levels, so a range scan keeps going up and down. **Idea:** keep all keys in the leaves and chain the leaves left to right; internal nodes only *route* the search, so more keys fit per page and the tree is even

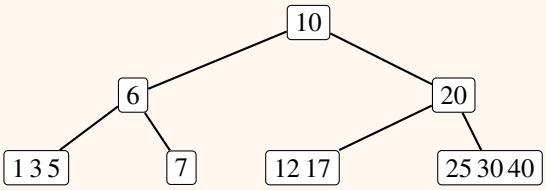
shorter.

Definition (B+ tree of order M)

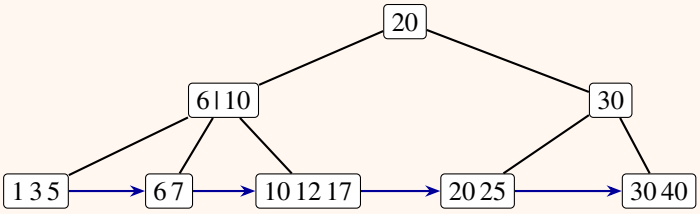
All keys (and records / record pointers) are stored in the **leaves**. Internal nodes store **separator keys** (copies that guide the search). Leaves are linked by a **next** pointer. All leaves are at the same depth; nodes are at least half full. In the notes $M = 4$: internal nodes have 2–4 children, leaves hold 2–3 keys.

From your notes: same keys: B-tree vs B+ tree

B-tree ($t = 2$): each key appears once, at any level



B+ tree ($M = 4$): every key is in a leaf; separators are copies; leaves linked



2.3.1 Node, search, range query

```c
#define M 4                             /* max children of an internal node    */
#define MAXK (M - 1)                    /* max keys in any node (here 3)       */
#define MIN_LEAF ((MAXK + 1) / 2)       /* min keys in a leaf          (2)     */
#define MIN_INT  ((M + 1) / 2 - 1)      /* min keys in internal node   (1)     */

typedef struct BPNode {
    bool leaf;
    int n;                              /* number of keys                      */
    int keys[MAXK + 1];                 /* +1 slot: temporary overflow         */
    struct BPNode *c[M + 1];            /* children (internal nodes only)      */
    struct BPNode *next;                /* next leaf (leaves only)             */
} BPNode;

BPNode *new_node(bool leaf) {
    BPNode *x = calloc(1, sizeof(BPNode));
    x->leaf = leaf;
    return x;
}
```

Search at each internal node go right while $k \geq$ separator; every search ends in a leaf, so *all searches cost exactly $h + 1$ node reads*.

```c
/* ---------------- SEARCH: always ends in a leaf ---------------- */
BPNode *find_leaf(BPNode *x, int k) {
    while (!x->leaf) {
        int i = 0;
        while (i < x->n && k >= x->keys[i]) i++;      /* keys >= separator go right */
        x = x->c[i];
    }
    return x;
}
bool bpt_search(BPNode *root, int k) {
```

```
    BPNode *leaf = find_leaf(root, k);
    for (int i = 0; i < leaf->n; i++) if (leaf->keys[i] == k) return true;
    return false;
}
```

Range query $[lo, hi]$: (1) descend once to the leaf that would hold lo; (2) follow `next` pointers, output keys until a key $> hi$. Cost: $\mathcal{O}(\log n)$ to reach the first leaf + one page per leaf scanned.

```
/* ---------------- RANGE QUERY [lo, hi] ---------------- */
void bpt_range(BPNode *root, int lo, int hi) {
    BPNode *leaf = find_leaf(root, lo);              /* 1. descend once        */
    while (leaf) {                                   /* 2. walk the leaf chain */
        for (int i = 0; i < leaf->n; i++) {
            if (leaf->keys[i] > hi) return;
            if (leaf->keys[i] >= lo) printf("%d ", leaf->keys[i]);
        }
        leaf = leaf->next;
    }
}
```

From your notes: range query $[6, 25]$ on the tree above

Descend: root 20: $6 < 20 \rightarrow$ left; node $[6 \mid 10]$: $6 \geq 6$ and $6 < 10 \rightarrow$ middle child $[6\,7]$. Output 6, 7; follow the chain: $[10\,12\,17] \rightarrow 10, 12, 17$; $[20\,25] \rightarrow 20, 25$; next leaf $[30\,40]$ starts with $30 > 25$: stop. Result 6 7 10 12 17 20 25.

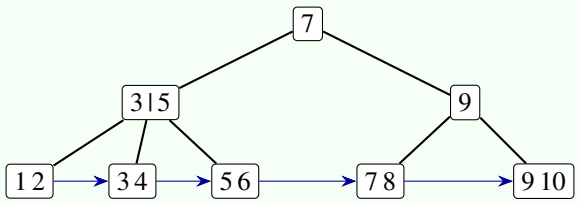
2.3.2 Insertion (supplement)

Insert into the correct leaf. If the leaf overflows (M keys), **split** it into two halves:

- **Leaf split: copy up.** The first key of the right half is *copied* into the parent (it stays in the leaf, because leaves must contain all keys). Link the new leaf into the chain.
- **Internal split: move up.** The middle key is *moved* to the parent (it leaves the node, like in a B-tree).
- If the root splits, a new root is created and the height grows by one.

Simple example: $M = 4$, insert $1, 2, \ldots, 10$

Leaves hold at most 3 keys. $1, 2, 3 \rightarrow [1\,2\,3]$. Insert 4: overflow $[1\,2\,3\,4] \rightarrow [1\,2][3\,4]$, copy 3 up: root $[3]$. Insert $5, 6$: $[3\,4\,5]$, then overflow $\rightarrow [3\,4][5\,6]$, copy 5 up: root $[3\,5]$. Insert $7, 8$: 8 overflows $[5\,6\,7\,8] \rightarrow [5\,6][7\,8]$, root $[3\,5\,7]$. Insert 9, then 10: $[7\,8\,9\,10] \rightarrow [7\,8][9\,10]$, copy 9 up: root would be $[3\,5\,7\,9]$, which overflows (an internal split): *move* 7 up and split into $[3\,5]$ and $[9]$; a new root $[7]$ appears.



(Verified by the program.)

```
/* ---------------- INSERT ---------------- */
/* Returns true if x split; then *up is the key pushed up, *right the new node. */
static bool insert_rec(BPNode *x, int k, int *up, BPNode **right) {
    if (x->leaf) {
        int i = x->n - 1;
        while (i >= 0 && x->keys[i] > k) { x->keys[i + 1] = x->keys[i]; i--; }
        x->keys[i + 1] = k; x->n++;
```

```
        if (x->n <= MAXK) return false;
        BPNode *r = new_node(true);                    /* leaf split: COPY up    */
        int keep = (x->n + 1) / 2;
        for (int j = keep; j < x->n; j++) r->keys[r->n++] = x->keys[j];
        x->n = keep;
        r->next = x->next; x->next = r;
        *up = r->keys[0]; *right = r;
        return true;
    }
    int i = 0;
    while (i < x->n && k >= x->keys[i]) i++;
    int cu; BPNode *cr;
    if (!insert_rec(x->c[i], k, &cu, &cr)) return false;
    for (int j = x->n; j > i; j--) { x->keys[j] = x->keys[j - 1]; x->c[j + 1] = x->c[
    j]; }
    x->keys[i] = cu; x->c[i + 1] = cr; x->n++;
    if (x->n <= MAXK) return false;
    BPNode *r = new_node(false);                       /* internal split: MOVE up */
    int mid = x->n / 2;
    *up = x->keys[mid];
    for (int j = mid + 1; j < x->n; j++) r->keys[r->n++] = x->keys[j];
    for (int j = mid + 1; j <= x->n; j++) r->c[j - mid - 1] = x->c[j];
    x->n = mid;
    *right = r;
    return true;
}
BPNode *bpt_insert(BPNode *root, int k) {
    int up; BPNode *r;
    if (insert_rec(root, k, &up, &r)) {               /* root split: tree grows  */
        BPNode *s = new_node(false);
        s->keys[0] = up; s->n = 1; s->c[0] = root; s->c[1] = r;
        return s;
    }
    return root;
}
```

2.3.3 Deletion (supplement)

Remove the key from its leaf. If the leaf has fewer than $\lceil (M-1)/2 \rceil$ keys: (1) **borrow** one key from a sibling (and fix the separator in the parent), else (2) **merge** with a sibling and delete the separator from the parent, which may in turn underflow. Separators that equal a deleted key may stay: they remain valid routing keys. If the root is left with no key, its only child becomes the new root.

```
/* ---------------- DELETE ---------------- */
static void remove_at(BPNode *x, int i) {              /* drop keys[i] and c[i+1] */
    for (int j = i + 1; j < x->n; j++) { x->keys[j - 1] = x->keys[j]; x->c[j] = x->c[
    j + 1]; }
    x->n--;
}
static void fix_child(BPNode *x, int i) {              /* c[i] may be under-full  */
    BPNode *c = x->c[i];
    int minc = c->leaf ? MIN_LEAF : MIN_INT;
    if (c->n >= minc) return;
    BPNode *L = i > 0 ? x->c[i - 1] : NULL, *R = i < x->n ? x->c[i + 1] : NULL;
    if (L && L->n > minc) {                            /* borrow from left sibling */
        for (int j = c->n; j > 0; j--) c->keys[j] = c->keys[j - 1];
        if (c->leaf) {
            c->keys[0] = L->keys[--L->n]; c->n++;
            x->keys[i - 1] = c->keys[0];
        } else {
            for (int j = c->n + 1; j > 0; j--) c->c[j] = c->c[j - 1];
            c->keys[0] = x->keys[i - 1]; c->c[0] = L->c[L->n];
            x->keys[i - 1] = L->keys[--L->n]; c->n++;
```

```
        }
    } else if (R && R->n > minc) {                    /* borrow from right sibling */
        if (c->leaf) {
            c->keys[c->n++] = R->keys[0];
            for (int j = 1; j < R->n; j++) R->keys[j - 1] = R->keys[j];
            R->n--; x->keys[i] = R->keys[0];
        } else {
            c->keys[c->n] = x->keys[i]; c->c[c->n + 1] = R->c[0]; c->n++;
            x->keys[i] = R->keys[0];
            for (int j = 1; j < R->n; j++) R->keys[j - 1] = R->keys[j];
            for (int j = 1; j <= R->n; j++) R->c[j - 1] = R->c[j];
            R->n--;
        }
    } else {                                          /* merge with a sibling    */
        int li = L ? i - 1 : i;                       /* merge c[li] and c[li+1] */
        BPNode *a = x->c[li], *b = x->c[li + 1];
        if (!a->leaf) a->keys[a->n++] = x->keys[li]; /* internal: pull separator down
     */
        for (int j = 0; j < b->n; j++) a->keys[a->n + j] = b->keys[j];
        if (!a->leaf) for (int j = 0; j <= b->n; j++) a->c[a->n + j] = b->c[j];
        a->n += b->n;
        if (a->leaf) a->next = b->next;
        free(b);
        remove_at(x, li);
    }
}
static void delete_rec(BPNode *x, int k) {
    if (x->leaf) {
        for (int i = 0; i < x->n; i++)
            if (x->keys[i] == k) {
                for (int j = i + 1; j < x->n; j++) x->keys[j - 1] = x->keys[j];
                x->n--; return;
            }
        return;
    }
    int i = 0;
    while (i < x->n && k >= x->keys[i]) i++;
    delete_rec(x->c[i], k);
    fix_child(x, i);
}
BPNode *bpt_delete(BPNode *root, int k) {
    delete_rec(root, k);
    if (!root->leaf && root->n == 0) { BPNode *o = root; root = root->c[0]; free(o);
    }
    return root;
}
```

2.4 B-tree vs. B+ tree

	B-tree	B+ tree
Where keys / data live	every node	leaves only
Internal nodes	keys + data	separator copies only
Duplicate keys in tree	no	yes (separators)
Search ends at	any level	always a leaf
Range / sequential scan	in-order traversal	follow the leaf chain
Fan-out per page	lower	higher

Where used: database indexes (B+ trees are the default index in relational databases), file systems (directory and extent indexes), key-value stores and embedded databases — anywhere data sits on block storage and one node equals one block (Comer, *The Ubiquitous B-Tree*, 1979).

Quiz answers (from the slides)

Q1 max keys per node: $2t - 1$ (C). Q2 nodes large so one node fills one disk page and the tree stays shallow (B). Q3 height grows only when the root splits (B). Q4 a node is split when a full node is met on the way down (B). Q5 root is $[8]$ (A). Q6 B+ trees: all keys in linked leaves, descend once then follow next pointers (B).

Chapter 3

Hashing

3.1 Can we search in constant time?

Data structure	Search	Insert	Delete
Unsorted array	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Sorted array	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Balanced search tree	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Array indexed by key	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Hash table	$\mathcal{O}(1)$ *average*	$\mathcal{O}(1)$ avg.	$\mathcal{O}(1)$ avg.

Question: can we look up any key in $\mathcal{O}(1)$ time using space proportional to the number of keys stored?

Idea: labelled drawers. Keep many small collections called *buckets*, like the drawers of a dresser. A rule says which drawer each item belongs in; to find an item, open only its drawer. The rule is a **hash function**: bucket $=$ hash(key) mod (number of buckets). Example: hash("urania") $= 12206$ and $12206 \bmod 6 = 2$, so "urania" goes into bucket 2.

What is a hash function?

A function that maps any input (number, string, file) to a value in a fixed range, usually an integer. It must be **deterministic** (same input $\Rightarrow$ same output) and should **spread inputs out** (different, even very similar, inputs usually give different outputs).

A hash function in code (from the notes): each letter is mixed in by multiplying by a prime and adding; taking the remainder keeps the value in range.

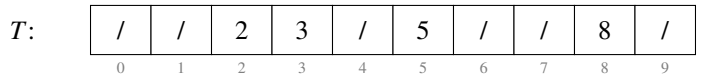
```
unsigned name_hash(const char *first, const char *last) {
    const unsigned kLargePrime = 16908799;
    const unsigned kSmallPrime = 127;
    unsigned h = 0;
    for (const char *s = first; *s; s++)        // letters of first name
        h = (kSmallPrime * h + tolower(*s)) % kLargePrime;
    for (const char *s = last; *s; s++)         // then of last name
        h = (kSmallPrime * h + tolower(*s)) % kLargePrime;
    return h;                                   // 0 .. kLargePrime - 1
}
```

Where hashing is used. Programming languages (Python `dict`/`set`, Java `HashMap`, C++ `unordered_map`); compilers (symbol tables); databases (hash indexes, hash joins); caches and memoisation; spell checkers. Beyond tables: password storage (store the hash, not the password), integrity checks (compare hashes of files/messages), fingerprints (songs, documents), finding repeats (duplicate files, plagiarism, common substrings).

3.2 The dictionary problem and direct addressing

A dynamic set supporting only INSERT(T,x), SEARCH(T,k), DELETE(T,x). **Goal:** all three in $\mathcal{O}(1)$ average time.

Direct-address table. Keys come from a small universe $U = \{0, \ldots, 9\}$; slot k holds the element with key k (empty slots hold NIL). Every operation is $\mathcal{O}(1)$ *worst case*.



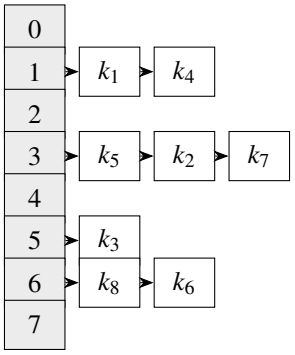
Problem: a direct-address table needs one slot per *possible* key. For 64-bit keys that is $\approx 1.8 \times 10^{19}$ slots, although we may store only 10,000 student IDs. **Fix:** use a table with only m slots and compute the slot with a hash function $h : U \to \{0, \ldots, m-1\}$; key k is stored in slot $h(k)$.

Collisions are unavoidable.

- *Pigeonhole principle:* there are more possible keys than slots, so two keys must share a slot.
- *Birthday paradox:* with just 23 people the chance that two share a birthday is 50.7%. Collisions appear long before the table is full (proved below).

Two ways to resolve collisions: **chaining** (each slot holds a linked list of all keys hashing there) and **open addressing** (all keys live in the table; on a collision, probe another slot).

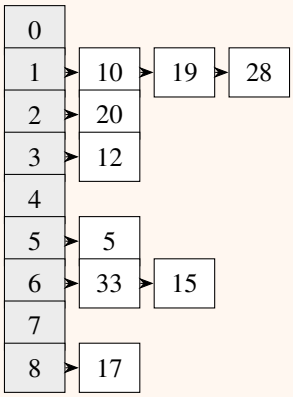
3.3 Chaining



Operation	What it does	Time
INSERT(T,x)	insert x at the head of list $T[h(x.key)]$	$\mathcal{O}(1)$
SEARCH(T,k)	search key k in list $T[h(k)]$	length of the list
DELETE(T,x)	remove x from list $T[h(x.key)]$	$\mathcal{O}(1)$ with a doubly linked list and a pointer to x

From your notes: chaining, $m = 9$, $h(k) = k \bmod 9$

Insert $5, 28, 19, 15, 20, 33, 12, 17, 10$ (new keys go to the *head*): $5 \to 5$; $28 \to 1$; $19 \to 1$ (collision, goes in front of 28); $15 \to 6$; $20 \to 2$; $33 \to 6$ (in front of 15); $12 \to 3$; $17 \to 8$; $10 \to 1$ (in front).



9 keys in 9 slots; longest list: slot 1 with 3 keys. **Searching:** search 19: slot 1, compare 10, then 19 found (2 comparisons). Search 28: slot 1, compare 10, 19, 28 (3). Search 11: slot 2, compare 20, not found (1). Cost = length of one list.

3.3.1 Load factor and simple uniform hashing

Load factor

$\alpha = n/m$, the average number of keys per slot (n keys, m slots).
Simple uniform hashing assumption (SUHA): every key is equally likely to hash to any of the m slots, independently of where the other keys went. It models h as a random function; it is an assumption about keys and h, not a theorem (for any fixed h some key sets break it). Open addressing uses a stronger version, *uniform hashing*: every probe sequence (permutation of the slots) is equally likely.

Lemma 1: every list has $\alpha = n/m$ keys on average

Fix slot j. Let $X_i = 1$ if key k_i hashes to j (else 0). Then $n_j = \sum_{i=1}^{n} X_i$ and $E[X_i] = \Pr[h(k_i) = j] = 1/m$. By linearity of expectation $E[n_j] = n/m = \alpha$. □

Lemma 2: two different keys collide with probability $1/m$

$\Pr[h(k) = h(l)] = \sum_{j=0}^{m-1} \Pr[h(k) = j] \Pr[h(l) = j] = m \cdot \frac{1}{m^2} = \frac{1}{m}$. By linearity over all pairs, the expected number of colliding pairs is $\binom{n}{2}\frac{1}{m}$. For $n = m = 1000$ ($\alpha = 1$) that is ≈ 500 colliding pairs. □

How many slots stay empty?

Slot j stays empty if each of the n keys independently misses it: probability $(1 - \frac{1}{m})^n \approx e^{-n/m} = e^{-\alpha}$. Summing indicators over the m slots, the expected fraction of empty slots is $e^{-\alpha}$:

α	0.5	1	2
expected empty slots	$\approx 60.7\%$	$\approx 36.8\%$	$\approx 13.5\%$

Collisions come early (birthday bound)

Claim: with n keys in m slots the probability of *no* collision is at most $e^{-n(n-1)/(2m)}$. Insert keys one at a time: key $i+1$ must miss the i slots already used, probability $1 - \frac{i}{m}$. Using $1 - x \le e^{-x}$,

$$\Pr[\text{no collision}] = \prod_{i=0}^{n-1}\left(1 - \frac{i}{m}\right) \le \prod_{i=0}^{n-1} e^{-i/m} = e^{-n(n-1)/(2m)}.$$

For $m = 365$, $n = 23$: $e^{-253/365} \approx 0.50$, so a shared birthday has probability about $1/2$ (exactly 50.7%).

In general collisions appear once $n \approx \sqrt{m}$. □

3.3.2 Search time with chaining

Theorem

Under simple uniform hashing a search (successful or not) takes $\Theta(1+\alpha)$ time on average: 1 to compute $h(k)$, α to scan the list. Keep m proportional to n so that $\alpha = \mathcal{O}(1)$ and every operation is $\mathcal{O}(1)$ on average.

Unsuccessful search

Let n_j be the length of list $T[j]$, $\sum_j n_j = n$, $E[n_j] = n/m$. A key k not in the table is equally likely to hash to any slot and the search scans the whole list, so the expected cost is $E[n_{h(k)}] = \frac{1}{m}\sum_j n_j = \frac{n}{m} = \alpha$. Adding $\mathcal{O}(1)$ for computing $h(k)$ gives $\Theta(1+\alpha)$. □

Successful search

Assume the searched key is equally likely to be any stored key $k_1, \ldots, k_n$ (in insertion order). Since new keys go to the *head*, searching for k_i examines k_i itself plus the keys inserted *after* it into the same list; each of the $n-i$ later keys lands in that list with probability $1/m$. Expected examined: $1 + \frac{n-i}{m}$. Averaging over i:

$$\frac{1}{n}\sum_{i=1}^{n}\left(1+\frac{n-i}{m}\right) = 1+\frac{n-1}{2m} = 1+\frac{\alpha}{2}-\frac{\alpha}{2n} = \Theta(1+\alpha). \quad \square$$

3.3.3 Chaining in C (with rehashing)

When α grows too large, *rehash*: allocate a bigger table (typically double) and re-insert every key (the hash value depends on m).

```c
/* ============ 1. CHAINING ============ */
typedef struct Node { int key; struct Node *next; } Node;
Node **T; int M;                        /* M buckets, calloc'd, all NULL */
static int h(int key) { return key % M; }

void chained_insert(int key) {          /* insert at head: O(1)          */
    Node *x = malloc(sizeof(Node));
    x->key = key;
    x->next = T[h(key)];
    T[h(key)] = x;
}
Node *chained_search(int key) {         /* walk one list                 */
    for (Node *x = T[h(key)]; x; x = x->next)
        if (x->key == key) return x;
    return NULL;
}
void chained_delete(int key) {          /* O(1) after the search with a doubly linked
    list;
                                           here singly linked: O(list length) */
    Node **pp = &T[h(key)];
    while (*pp && (*pp)->key != key) pp = &(*pp)->next;
    if (*pp) { Node *d = *pp; *pp = d->next; free(d); }
}
void rehash(void) {                     /* double the number of buckets  */
    int old_m = M;
    Node **old = T;
    M = 2 * old_m;                      /* h(key) now uses the new M     */
    T = calloc(M, sizeof(Node *));
```

```
    for (int i = 0; i < old_m; i++) {
        Node *x = old[i];
        while (x != NULL) {            /* move each node to its new list */
            Node *next = x->next;
            x->next = T[h(x->key)];
            T[h(x->key)] = x;
            x = next;
        }
    }
    free(old);
}
void print_chains(void) {
    for (int i = 0; i < M; i++) {
        printf("  %d:", i);
        for (Node *x = T[i]; x; x = x->next) printf(" %d ->", x->key);
        printf(" NIL\n");
    }
}
```

3.4 Open addressing

Every slot holds at most one key. To insert, **probe** slots in a fixed order $h(k,0), h(k,1), \ldots$ until an empty one is found; search follows the same order. An empty slot ends the search (the key would have been placed there).

Algorithm 6: Hash-Insert(T,k)

```
i ← 0;
repeat
    j ← h(k,i);
    if T[j] = NIL or T[j] = DELETED then
        T[j] ← k; return j
    else
        i ← i+1
until i = m;
error "hash table overflow";
```

Algorithm 7: Hash-Search(T,k)

```
i ← 0;
repeat
    j ← h(k,i);
    if T[j] = k then return j;
    i ← i+1
until T[j] = NIL or i = m;
return NIL;
```

Probe sequence requirement: for every key, $h(k,0), \ldots, h(k,m-1)$ must be a permutation of $0, \ldots, m-1$ (otherwise an insert can fail while empty slots remain). The ideal *uniform hashing* makes every one of the $m!$ permutations equally likely; real schemes approximate it.

3.4.1 Three probe sequences

Scheme	$h(k,i)$	Distinct sequences
Linear probing	$(h'(k)+i) \bmod m$	m
Quadratic probing	$(h'(k)+c_1 i+c_2 i^2) \bmod m$	m
Double hashing	$(h_1(k)+i\,h_2(k)) \bmod m$	$\approx m^2$

For double hashing: choose m a power of 2 and h_2 always odd, or m prime and $1 \le h_2(k) < m$ (so $h_2(k)$ is coprime to m and the sequence visits every slot).

From your notes: linear probing, $m = 11$, $h(k) = k \bmod 11$

Insert $10, 22, 31, 4, 15, 28, 17, 88, 59$:
$10 \to 10$; $22 \to 0$; $31 \to 9$; $4 \to 4$; $15 \to 4$ occupied, $\to 5$ (1 collision); $28 \to 6$; $17 \to 6$ occupied, $\to 7$; $88 \to 0$ occupied, $\to 1$; $59 \to 4, 5, 6, 7$ occupied, $\to 8$ (4 collisions).

slot:	22	88			4	15	28	17	59	31	10
	0	1	2	3	4	5	6	7	8	9	10

3.4.2 Clustering

Primary clustering (linear probing): long runs of occupied slots build up; any key hashing into a run walks to its end and *lengthens* it. In a run of i full slots preceded by an empty slot, that empty slot is filled next with probability $(i+1)/m$ – runs grow faster the longer they are. **Secondary clustering** (quadratic probing): keys with the same home slot follow the same probe sequence. Remedy: double hashing, where the step depends on the key.

From your notes: 3, 14, 25, 36, 47 (home slot 3), then 4, 15 (home slot 4); $m = 11$

Linear: probes $1, 2, 3, 4, 5, 5, 6 \Rightarrow$ **26 probes** (the notes list the probe counts per key); quadratic ($c_1 = c_2 = 1$): **20 probes**; double hashing with $h_2(k) = 1 + (k \bmod 10)$: **13 probes**.

linear (26):				3	14	25	36	47	4	15	
	0	1	2	3	4	5	6	7	8	9	10

quadratic (20):		47		3	36	14	4			25	15

double (13):	47			3	4	15			14	25	36

The linear table forms one run of 7 keys: keys 4 and 15 never collided with each other at home yet paid 5 and 6 probes. For a half-full table ($\alpha = \frac{1}{2}$) the average cost of a failed search is 1.5 probes for spread-out keys but 3.25 for $m = 16$ (and ≈ 126 for $m = 1000$) if the keys form one run: same load, very different cost.

Why quadratic probing visits only some slots

c_1, c_2, m must be chosen so that every slot is visited. E.g. with m a power of 2, $h(k,i) = h'(k) + \frac{i+i^2}{2}$ visits all slots; with $m = 11$ and $c_1 = c_2 = 1$ the sequence of the key with home slot 3 is $3, 5, 9, 4, 1, \ldots$ (as in the notes).

3.4.3 Deletion

Never reset a deleted slot to NIL: a later search for a key further along the probe sequence would stop at the hole and wrongly report "not found". Store a special value DELETED instead: search skips it, insert may reuse it.

wrong:	22	88			4		28	17	59	31	10

right:	22	88			4	DEL	28	17	59	31	10

Deleting 15 (slot 5) and searching 59 (probes 4,5,6,7,8): with an empty slot 5 the search stops at 5 (not found); with DELETED it continues and finds 59 in slot 8. **Consequence:** with many deletions, DELETED marks accumulate and searches slow down; rehash periodically (chaining avoids this problem).

Theorem (unsuccessful search, open addressing)

With uniform hashing and $\alpha = n/m < 1$, an unsuccessful search makes at most $\dfrac{1}{1-\alpha}$ probes on average. (E.g. $\alpha = 0.5 \Rightarrow 2$ probes; $\alpha = 0.9 \Rightarrow 10$.) Insertion costs the same.

Proof

Let X be the number of probes. $X \geq i$ exactly when the first $i-1$ probes all hit full slots. The first probe hits one of n full slots among m; the next one of $n-1$ among the remaining $m-1$; and so on:

$$\Pr[X \geq i] = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdots \frac{n-i+2}{m-i+2} \leq \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1}.$$

Hence $E[X] = \sum_{i \geq 1} \Pr[X \geq i] \leq \sum_{i \geq 1} \alpha^{i-1} = \frac{1}{1-\alpha}$. Insertion is an unsuccessful search followed by one write. □

(For a *successful* search the bound is $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$, e.g. ≤ 1.39 probes at $\alpha = \frac{1}{2}$ and ≤ 2.56 at $\alpha = 0.9$.)

3.4.4 Open addressing in C

```c
/* ============ 2. OPEN ADDRESSING ============ */
#define EMPTY   (-1)
#define DELETED (-2)
enum { LINEAR, QUADRATIC, DOUBLE };

int OM = 11;                            /* table size                    */
int *OT;                                /* the table                     */
int mode = LINEAR;

/* probe sequence h(k,i) */
static int probe(int k, int i) {
    int h1 = k % OM;
    switch (mode) {
    case LINEAR:    return (h1 + i) % OM;
    case QUADRATIC: return (h1 + i + i * i) % OM;          /* c1 = c2 = 1       */
    default:        return (h1 + i * (1 + k % (OM - 1))) % OM; /* h2 = 1 + k mod 10
    */
    }
}
/* returns slot, or -1 if the table is full; *probes = slots examined */
int oa_insert(int k, int *probes) {
    for (int i = 0; i < OM; i++) {
        int j = probe(k, i);
        if (OT[j] == EMPTY || OT[j] == DELETED) { OT[j] = k; *probes = i + 1; return
    j; }
    }
    return -1;                          /* hash table overflow           */
}
int oa_search(int k) {
    for (int i = 0; i < OM; i++) {
        int j = probe(k, i);
        if (OT[j] == k) return j;
        if (OT[j] == EMPTY) return -1;    /* DELETED slots do not stop us */
    }
    return -1;
}
void oa_delete(int k) {
```

```
    int j = oa_search(k);
    if (j >= 0) OT[j] = DELETED;      /* never reset to EMPTY          */
}
void oa_reset(int m, int md) {
    OM = m; mode = md; free(OT);
    OT = malloc(m * sizeof(int));
    for (int i = 0; i < m; i++) OT[i] = EMPTY;
}
void oa_print(void) {
    for (int i = 0; i < OM; i++) {
        if (OT[i] == EMPTY) printf("    . "); else if (OT[i] == DELETED) printf(" DEL
    "); else printf("%4d ", OT[i]);
    }
    printf("\n");
}
```

Time / space complexity

	Chaining	Open addressing
Search (avg.)	$\Theta(1+\alpha)$	$\leq \frac{1}{1-\alpha}$ (unsucc.)
Insert (avg.)	$\mathcal{O}(1)$	$\leq \frac{1}{1-\alpha}$
Delete	$\mathcal{O}(1)$ with doubly linked list	needs DELETED marks
Load factor	any α (even > 1)	$\alpha < 1$ required
Worst case (any scheme)	$\Theta(n)$	$\Theta(n)$

3.5 Hash functions

A good hash function is **uniform** (keys spread evenly), **deterministic**, **fast** (a few arithmetic operations) and **uses the whole key** (patterns in the keys must not cause collisions). **Keys as numbers:** the string "pt" becomes $112 \cdot 128 + 116 = 14452$ by reading ASCII codes as digits in base 128.

Division method: $h(k) = k \bmod m$. Examples with $m = 12$: $100 \bmod 12 = 4$, $25 \bmod 12 = 1$, $36 \bmod 12 = 0$. **Good** m: a prime not too close to a power of 2 (e.g. about 2000 strings, $m = 701$). **Bad:** $m = 2^p$ (then $h(k)$ is just the lowest p bits of k; the others are ignored) and $m = 10^d$ for decimal keys (only the last d digits matter).

Multiplication method: $h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$ for a constant $0 < A < 1$: multiply k by A, keep the fractional part, multiply by m, round down. Works for any m (often $m = 2^p$); Knuth suggests $A = (\sqrt{5}-1)/2 \approx 0.618$.

```
/* ============ 3. HASH FUNCTIONS ============ */
unsigned div_hash(unsigned k, unsigned m) { return k % m; }
unsigned mul_hash(unsigned k, unsigned m) {          /* Knuth: A = (sqrt5 - 1)/2 */
    const double A = 0.6180339887498949;
    double x = k * A;
    return (unsigned)(m * (x - (unsigned)x));
}
/* universal family  h_{a,b}(k) = ((a k + b) mod p) mod m  */
unsigned universal_hash(unsigned long a, unsigned long b, unsigned long p, unsigned m
    , unsigned long k) {
    return (unsigned)(((a * k + b) % p) % m);
}
```

3.6 Universal hashing

Danger. For any fixed hash function an adversary can pick keys that all land in one slot, making search $\Theta(n)$.

Claim: every fixed hash function has a bad input

If the universe has at least $N \cdot M$ keys, then for any h some N keys fall into the same bucket. *Pigeonhole:* if every bucket received at most $N-1$ keys of the universe, the universe would have fewer than $N \cdot M$ keys. □

So a guarantee for *every* input must come from choosing h **at random** from a family when the table is created.

Universal family

A family $\mathscr{H}$ of hash functions is **universal** if for any two different keys $x \neq y$, a random $h \in \mathscr{H}$ gives $\Pr[h(x) = h(y)] \leq 1/M$.
Theorem. For any set of N keys and a key x, the expected number of other keys colliding with x is at most $(N-1)/M < N/M$. (Proof: sum the indicator "$h(y) = h(x)$" over the other $N-1$ keys, each with expectation $\leq 1/M$.) So chaining has expected $\mathscr{O}(1+N/M)$ cost *for every input*.

3.6.1 Family 1: $h_{a,b}(k) = ((ak+b) \bmod p) \bmod m$

Pick a prime p larger than every key, then random $a \in \{1,\ldots,p-1\}$ and $b \in \{0,\ldots,p-1\}$.

From your notes: $p = 17$, $m = 6$

$h_{3,4}(8) = (3 \cdot 8 + 4 \bmod 17) \bmod 6 = (28 \bmod 17) \bmod 6 = 11 \bmod 6 = 5$.

k	0	1	2	3	4	5	6	7	8	9	10
$h_{3,4}(k)$	4	1	4	1	4	2	5	2	5	2	0
$h_{5,2}(k)$	2	1	0	0	5	4	3	3	2	1	1

(Both rows are reproduced by the C code `universal_hash`.)

Why it is universal

Fix keys $k \neq l$ (both $< p$). Let $r = (ak+b) \bmod p$, $s = (al+b) \bmod p$. Then $r - s \equiv a(k-l) \not\equiv 0 \pmod p$ (since p is prime and $a, k-l \neq 0$), so $r \neq s$. Moreover the map $(a,b) \mapsto (r,s)$ is a bijection between the $p(p-1)$ choices of (a,b) and the ordered pairs $r \neq s$. A collision means $r \equiv s \pmod m$. For a given r, the number of $s \neq r$ in $\{0,..,p-1\}$ with $s \equiv r \pmod m$ is at most $\lceil p/m \rceil - 1 \leq (p-1)/m$. Hence

$$\Pr[\text{collision}] \leq \frac{p \cdot (p-1)/m}{p(p-1)} = \frac{1}{m}. \quad \square$$

3.6.2 Family 2: dot product modulo a prime

Split a key into pieces $x = (x_1,\ldots,x_k)$, each $< M$ (M prime). Pick random $a_1,\ldots,a_k \in \{0..M-1\}$ and let $h_a(x) = \left(\sum_i a_i x_i\right) \bmod M$.

From your notes

$M = 7$, $a = (3,5,1)$: $h(2,4,6) = (6+20+6) \bmod 7 = 32 \bmod 7 = 4$.

Why universal (collision probability exactly $1/M$)

If $x \neq y$ differ in piece i, a collision means $\sum_j a_j(x_j - y_j) \equiv 0 \pmod M$. Fix all a_j with $j \neq i$: the equation becomes $a_i(x_i - y_i) \equiv c \pmod M$. Because M is prime and $x_i - y_i \not\equiv 0$ it has an inverse, so exactly one value of a_i out of M works. □

3.7 Perfect hashing (static sets)

For a *fixed* set of n keys we can search in $\mathcal{O}(1)$ **worst case**. **Level 1:** hash the n keys into n slots with a universal function. **Level 2:** a slot with n_j keys gets its *own* table of size n_j^2 with a hash function chosen without collisions. Lookup = two hash computations.

From your notes: keys 10, 22, 37, 40, 52, 60, 70, 72, 75

Level 1: k mod 9 gives slot $0:\{72\}$, $1:\{10,37\}$, $3:\{75\}$, $4:\{22,40\}$, $6:\{60\}$, $7:\{52,70\}$. Slots 1, 4, 7 hold two keys, so they get size-4 tables using k mod 4: $10\to 2$, $37\to 1$; $22\to 2$, $40\to 0$; $52\to 0$, $70\to 2$ – no collisions.

Why size n_j^2 and why total space is $\mathcal{O}(n)$

Level 2 succeeds quickly. With a universal h_j into n_j^2 slots, the expected number of colliding pairs is $\binom{n_j}{2}\cdot\frac{1}{n_j^2}<\frac{1}{2}$. By Markov's inequality $\Pr[\text{some collision}]<\frac{1}{2}$, so each table needs expected <2 random tries.
Total space. $\sum_j n_j^2=\sum_j\left(n_j+2\binom{n_j}{2}\right)=n+2\cdot\#\{\text{colliding pairs at level 1}\}$, whose expectation is $\le n+2\binom{n}{2}\frac{1}{n}=2n-1<2n$. So expected space is $\mathcal{O}(n)$. (Fredman–Komlós–Szemerédi, 1984.) □

3.8 Solved quiz questions

1. **Linear probing**, $m=11$, keys $20,31,42,9,53$ (table empty), probes counting the final empty slot: $20\to 9$ (1); $31\to 9,10$ (2); $42\to 9,10,0$ (3); $9\to 9,10,0,1$ (4); $53\to 9,10,0,1,2$ (5). Total = **15**.
2. **Double hashing**, $m=11$, $h_1=k$ mod 11, $h_2=1+(k \bmod 10)$; table after $14,25,36$ ($14\to 3$; $25\to 3$ taken, step 6: 9; $36\to 3$ taken, step 7: 10). Insert 47: $h_1=3$ taken, step $1+7=8$: $(3+8) \bmod 11=0$ free $\Rightarrow$ **slot 0**.
3. **Why mark DELETED instead of NIL?** So a search does not stop early and miss keys further along the probe sequence: **C**.
4. **FKS, $n_j=4$ keys in slot j:** secondary table size $n_j^2=16$: **C**.
5. **20 keys, 20 slots, expected empty slots:** each slot is empty with probability $(19/20)^{20}$, so $20\cdot(19/20)^{20}$ (about 7.2): **C**.
6. **Chaining, $m=7$:** slot 1 holds $22\to 8$; insert 43 ($43 \bmod 7=1$) at the head, then 29 ($29 \bmod 7=1$) at the head: chain $29\to 43\to 22\to 8$. A successful search for 8 makes **4** comparisons.
7. **Double hashing with $m=16$:** for every probe sequence to visit all slots, h_2 must be coprime to 16, i.e. odd: $h_2(k)=2(k \bmod 8)+1$ **(C)**.

Chapter 4

Bloom Filters

4.1 The membership problem

Given a set S of n items, answer quickly: *is $x \in S$?* Examples: is this username taken? Is this URL on a blocklist? Is this key in this disk file? An exact set must store the keys: 10^9 URLs $\times$ 50 bytes $\approx$ 50 GB. A **Bloom filter** with a 1% error rate needs only $\approx$ 1.2 GB. We trade a small, tunable chance of a wrong "yes" for a large saving in space.

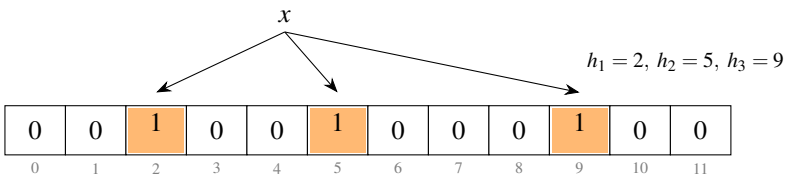
4.2 Structure: m bits and k hash functions

Bloom filter (Bloom, 1970)

A bit array $B[0..m-1]$ (all 0 at the start) and k hash functions $h_1, \ldots, h_k$ mapping items to $\{0, \ldots, m-1\}$.

- **Insert**(x): set $B[h_i(x)] = 1$ for $i = 1..k$ (never checks anything).
- **Query**(x): if *any* $B[h_i(x)] = 0$ answer **"definitely not in S"**; if all are 1 answer **"probably in S"**.

The filter never stores x itself, only these bits. Bits are only ever set, never cleared. Both operations take $\mathcal{O}(k)$ time however many items are stored.



Pseudocode

Algorithm 8: BF-Insert(x)

```
for i ← 1 to k do
    B[h_i(x)] ← 1
```

Algorithm 9: BF-Query(x)

```
for i ← 1 to k do
    if B[h_i(x)] = 0 then
        return NO ▷ definitely absent
return MAYBE ▷ probably present
;
```

C code

Instead of k independent hash functions we derive them from two base hashes by $h_i(x) = h_1(x) + i \cdot h_2(x)$ (Kirsch–Mitzenmacher double hashing), which is as good in practice. The 64-bit finalizer `mix` spreads the

bits of FNV-1a (without it the measured false-positive rate was about half of the theory, because raw FNV is poorly mixed).

```c
typedef struct {
    int m, k;                   /* bits, hash functions   */
    unsigned char *bits;        /* m bits, all 0 at start */
} Bloom;

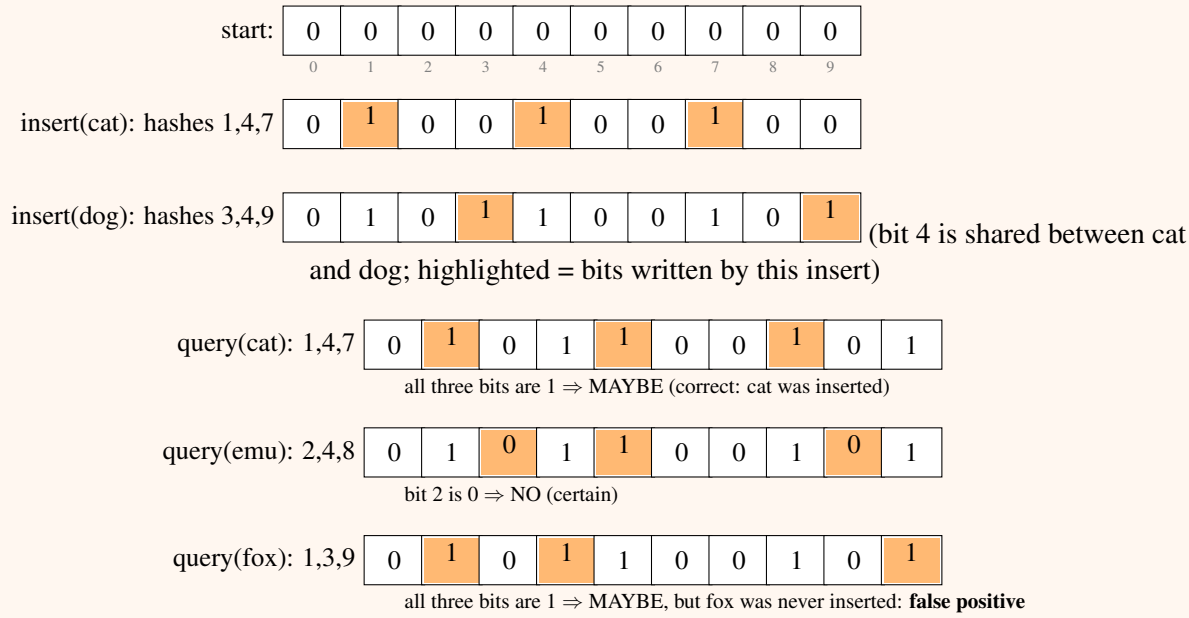
static void set_bit(unsigned char *b, uint64_t i) { b[i >> 3] |= (unsigned char)(1u << (i & 7)); }
static bool get_bit(const unsigned char *b, uint64_t i) { return (b[i >> 3] >> (i & 7)) & 1u; }

/* FNV-1a, followed by a 64-bit finalizer ("mix") so that all bits are well spread */
static uint64_t mix(uint64_t z) {
    z += 0x9e3779b97f4a7c15ULL;
    z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9ULL;
    z = (z ^ (z >> 27)) * 0x94d049bb133111ebULL;
    return z ^ (z >> 31);
}
static uint64_t fnv1a(const char *s, uint64_t seed) {
    uint64_t h = 14695981039346656037ULL ^ seed;
    for (; *s; s++) { h ^= (unsigned char)*s; h *= 1099511628211ULL; }
    return mix(h);
}
/* h_i(x) = h1 + i*h2  (double hashing, Kirsch-Mitzenmacher): k hashes from 2 base hashes */
static uint64_t h(int i, const char *x) {
    uint64_t h1 = fnv1a(x, 0), h2 = fnv1a(x, 0x9e3779b97f4a7c15ULL) | 1;
    return h1 + (uint64_t)i * h2;
}

Bloom *bf_create(int m, int k) {
    Bloom *f = malloc(sizeof(Bloom));
    f->m = m; f->k = k;
    f->bits = calloc((m + 7) / 8, 1);
    return f;
}
void bf_insert(Bloom *f, const char *x) {
    for (int i = 0; i < f->k; i++)
        set_bit(f->bits, h(i, x) % f->m);       /* B[h_i(x)] = 1   */
}
bool bf_query(Bloom *f, const char *x) {
    for (int i = 0; i < f->k; i++)
        if (!get_bit(f->bits, h(i, x) % f->m))
            return false;                       /* definitely not in S */
    return true;                                /* probably in S       */
}
/* union of two filters with the same m, k and hash functions: bitwise OR */
void bf_union(Bloom *a, const Bloom *b) {
    for (int i = 0; i < (a->m + 7) / 8; i++) a->bits[i] |= b->bits[i];
}
```

```c
/* ---- design helpers ---- */
int    bf_optimal_m(int n, double p) { return (int)ceil(-n * log(p) / (log(2) * log(2))); }
int    bf_optimal_k(int m, int n)    { return (int)round((double)m / n * log(2)); }
double bf_fp_rate(int m, int n, int k) { return pow(1.0 - exp(-(double)k * n / m), k); }
```

From your notes: $m = 10$, $k = 3$: insert cat, dog; query cat, emu, fox



4.3 One-sided error

	Filter says MAYBE	Filter says NO
x inserted	true positive	**impossible**
x not inserted	**false positive** (rate p)	true negative

- **No false negatives:** if x was inserted, all its k bits are 1 and bits are never cleared.
- **Some false positives:** other items may have set all k bits of a non-member.
- In practice, on MAYBE confirm with the exact (slower) store; a NO skips that work.

4.4 Analysis

Assume ideal hashing: each $h_i(x)$ is uniform on $\{0..m-1\}$ and independent.

Probability that a given bit is still 0

One hash call misses bit j with probability $1 - \frac{1}{m}$. n insertions make kn independent hash calls, all of which must miss:

$$\Pr[B_j = 0] = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m} \qquad \left(\text{using } (1 - \tfrac{1}{m})^m \to e^{-1}\right).$$

So about a fraction $e^{-kn/m}$ of the bits remain 0.

False-positive rate

Query a non-member y. Each of its k bits is 1 with probability $1 - e^{-kn/m}$. A false positive needs all k of them to be 1:

$$\boxed{p \approx \left(1 - e^{-kn/m}\right)^k}$$

More items $n \Rightarrow$ more bits set $\Rightarrow$ p rises; more bits $m \Rightarrow$ p falls.

Optimal number of hash functions

Fix m, n and write $q = e^{-kn/m}$, so $k = -\frac{m}{n}\ln q$. Then $\ln p = k\ln(1-q) = -\frac{m}{n}\ln q \ln(1-q)$, which is symmetric under $q \leftrightarrow 1-q$ and therefore minimal at $q = \frac{1}{2}$ (half the bits are set). That gives

$$k_{\text{opt}} = \frac{m}{n}\ln 2 \approx 0.693\,\frac{m}{n}, \qquad p_{\min} = 2^{-k_{\text{opt}}} = (0.6185)^{m/n}, \qquad m = -\frac{n\ln p}{(\ln 2)^2} \approx 1.44\, n \log_2 \frac{1}{p}.$$

Too many hash functions ($k \gg k_{\text{opt}}$) fill the array with 1s and p *increases* (quiz Q3).

Simple example: designing a filter

$n = 100{,}000$ items, target $p = 1\%$: $m = -n\ln 0.01/(\ln 2)^2 \approx 958{,}506$ bits ≈ 9.6 bits per item (≈ 117 KB), $k = \text{round}(9.59 \cdot 0.693) = 7$. Our program: theoretical $p = 1.004\%$, measured $p = 1.018\%$ on 200,000 non-members, and 0 false negatives among the 100,000 members. For 10^9 URLs: 9.6×10^9 bits ≈ 1.2 GB.

4.5 Operations a Bloom filter cannot do (and variants)

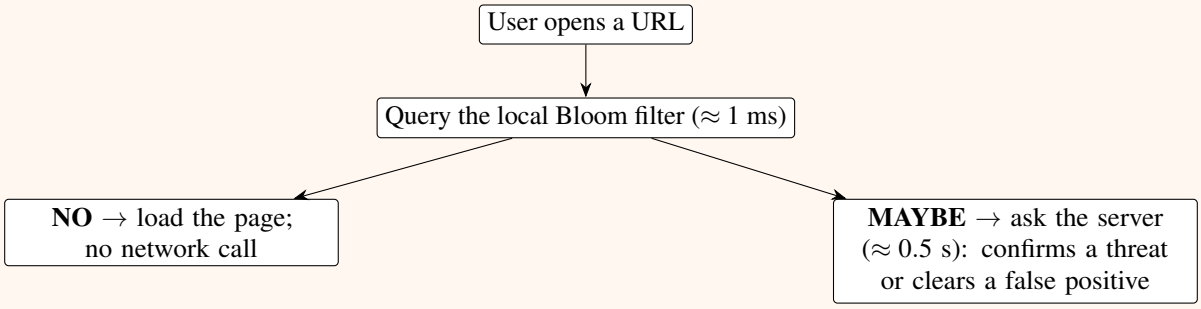
- **Delete:** setting an element's k bits back to 0 is wrong, because a bit may be shared with other elements, so clearing it could create false negatives (quiz Q2). Variant: *counting Bloom filter* (counters instead of bits; insert increments, delete decrements).
- **List the elements:** impossible, the filter stores no keys (quiz Q4).
- **Union:** two filters with the same m, k and hash functions for S_1, S_2: the bitwise OR is exactly the filter for $S_1 \cup S_2$ (quiz Q5). (Bitwise AND only approximates $S_1 \cap S_2$.) Implemented as `bf_union`.
- **Resize:** not possible without the original keys; choose m for the expected n.

4.6 Applications

- **Web browsers:** check URLs against a local filter of malicious sites; ask the server only on MAYBE.
- **Databases** (Bigtable, Cassandra, HBase, RocksDB): one filter per disk file skips reads for absent keys.
- **CDNs:** cache an object only on its second request (no "one-hit wonders").
- **Networks:** routers track blocked IP addresses; replicas exchange a filter instead of a full key list.
- Common pattern: a small in-memory filter in front of a slow, exact lookup.

From your notes: case study: Safe Browsing

Early Chrome kept a Bloom filter of known malicious URLs inside the browser; the full list stays on Google's servers.



Almost every URL is safe, so almost every check ends at NO without touching the network.

Time / space complexity

Insert / Query	$\mathcal{O}(k)$ time, independent of n
Space	$m \approx 1.44\, n \log_2(1/p)$ bits
False negatives	never
False positives	$\approx (1 - e^{-kn/m})^k$

Quiz answers

Q1 only false positives (C). Q2 a bit may be shared, so clearing it can cause false negatives (B). Q3 the rate increases because the array fills with 1s (C). Q4 list the elements (C). Q5 the OR is a filter for $S_1 \cup S_2$ (B).

Chapter 5

Heaps and Priority Queues

5.1 Priority queues

Sometimes items (e.g. processes) wait in a queue and must be served by *priority*. We do not need a full BST/red-black tree: we only need

- INSERT(x), DELETE-MAX() (remove and return the largest), BUILD from n keys (and MAX()).

If we have such a structure we can also **sort**: build the priority queue from the keys and successively delete the maximum.

Structure	Insert	Delete-Max	Max
Unsorted array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Sorted array	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
AVL tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
Binary heap	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$

We want a *simple* structure (no pointers, no rotations): the **binary heap**. We discuss max-heaps; everything translates to min-heaps by reversing the inequality.

5.2 Definition: two properties

Binary max-heap

Structural property: a *complete binary tree*: all levels are full except possibly the last, which is filled from the left.
Max-heap property: for every node i, $A[i] \geq A[j]$ for every child j of i (hence for every node in the subtree rooted at i).
A max-heap is an **array** satisfying both properties at all nodes. There is a bijection between arrays and binary trees of this shape, so the tree structure is *implicit*.

Heap property $\neq$ search property: there need not be any relation between the left and right siblings.

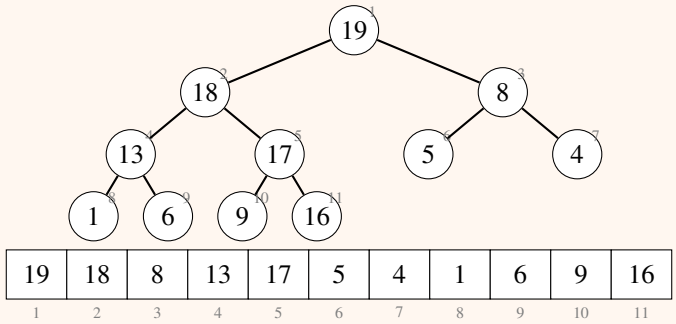
5.2.1 Array representation (1-indexed)

For the node stored at $A[i]$:

$$\text{left child} = A[2i], \qquad \text{right child} = A[2i+1], \qquad \text{parent} = A\big[\lfloor i/2 \rfloor\big].$$

(With 0-indexing: children $2i+1, 2i+2$, parent $\lfloor (i-1)/2 \rfloor$.)

From your notes: the heap drawn in the notes, $n = 11$



Check: $A[2] = 18$ has children $A[4] = 13$ and $A[5] = 17$; the parent of $A[7] = 4$ is $A[\lfloor 7/2 \rfloor] = A[3] = 8$.

5.2.2 Height and size

Height of a heap with n nodes

height $= \lfloor \log_2 n \rfloor$. Conversely a heap of height h has $2^h \le n \le 2^{h+1} - 1$ nodes.

Proof

Levels $0..h-1$ are full: $1 + 2 + \cdots + 2^{h-1} = 2^h - 1$ nodes, and the last level has between 1 and 2^h nodes. So $2^h \le n \le 2^{h+1} - 1$, hence $h \le \log_2 n < h + 1$. □

5.2.3 Consequences of the max-heap property

- The **largest** element is at the root (MAX is $\mathcal{O}(1)$).
- The **second largest** is a child of the root (otherwise it would violate the heap property with its parent, which would be larger).
- The **third largest** is a child of the root or a grandchild; if it is a grandchild, its parent is the second largest.
- Number of leaves $= \lceil n/2 \rceil$ (nodes $\lfloor n/2 \rfloor + 1..n$), so the last internal node is $\lfloor n/2 \rfloor$.

5.3 Insert (sift up)

Append x at the end of the array (the simplest place that keeps the tree complete): $A[n+1] = x$. This can violate the heap property only between x and its parent. **Fix by swapping with the parent, and repeat while the parent is smaller.**

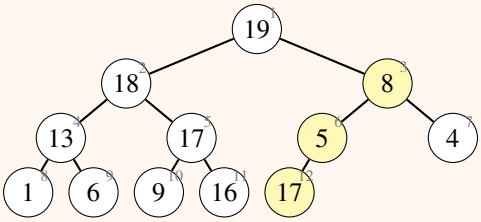
Algorithm 10: Heap-Insert(A, x)

```
1 n ← n + 1; A[n] ← x; j ← n;
2 while j > 1 and A[⌊j/2⌋] < A[j] do
3 |   swap A[⌊j/2⌋] ↔ A[j]; j ← ⌊j/2⌋
```

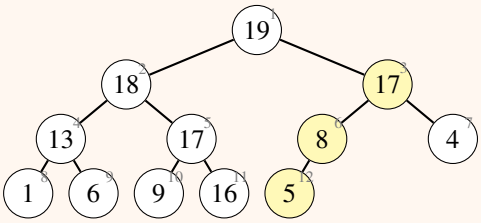
Cost: the number of swaps is at most the height, $\log n$, so **time** $\mathcal{O}(\log n)$. **Correctness:** a swap puts the larger value x above its parent p; p moves into x's old position, whose subtree only contains values that were descendants of p (all $\le p$), so the heap property holds below; only the link between x and its new parent remains to be checked.

From your notes: insert 17 into the heap above

Append 17 at position 12; its parent is position 6 (value 5): $17 > 5$, swap. Now at position 6, parent is position 3 (value 8): $17 > 8$, swap. Now at position 3, parent is position 1 (value 19): $17 < 19$, stop. Two swaps.



Highlighted: the path $12 \to 6 \to 3$ travelled by the new key. Final array: $19, 18, 17, 13, 17, 8, 4, 1, 6, 9, 16, 5$ (matches the C program).



Simple example: build a heap by inserting 3, 9, 5, 12

$[3]$; insert 9: $[3,9] \to$ swap $\to [9,3]$; insert 5: $[9,3,5]$ ok; insert 12 at position 4 (parent position 2 holds 3): swap $\to [9,12,5,3]$, parent position 1 holds 9: swap $\to [12,9,5,3]$.

5.4 Delete-Max and Max-Heapify (sift down)

Returning the maximum is $\mathcal{O}(1)$. But deleting the root destroys the structure, so **swap the root with the last element** and shrink the array. The new root may violate the heap property: **repeatedly replace it with the larger child** until it is no smaller than both children. This downward repair is MAX-HEAPIFY (it resolves a downward conflict).

Algorithm 11: Max-Heapify(A, n, i) (precondition: subtrees of i are heaps)

```
1 l ← 2i; r ← 2i + 1; largest ← i;
2 if l ≤ n and A[l] > A[largest] then largest ← l;
3 if r ≤ n and A[r] > A[largest] then largest ← r;
4 if largest ≠ i then
5 |   swap A[i] ↔ A[largest]; Max-Heapify(A, n, largest)
```

Algorithm 12: Heap-Delete-Max(A)

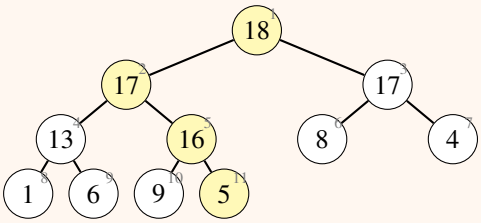
```
1 max ← A[1]; A[1] ← A[n]; n ← n − 1; Max-Heapify(A, n, 1); return max;
```

Time $\mathcal{O}(\log n)$ (at most one swap per level). **Why it is still a max-heap:** the replaced values are only *higher* in nature (the larger child moves up, over the other child, which is $\leq$ it), so the other subtree does not violate the heap property.

From your notes: delete-max on the heap above (after inserting 17)

Remove 19; move the last element (5) to the root: $[5, 18, 17, 13, 17, 8, 4, 1, 6, 9, 16]$. Sift down: children $18, 17 \Rightarrow$ swap with 18; at position 2 children $13, 17 \Rightarrow$ swap with 17 (position 5); at position 5 children

$9, 16 \Rightarrow$ swap with 16. Result: $18, 17, 17, 13, 16, 8, 4, 1, 6, 9, 5$ (matches the program).



5.5 Heap sort

Build a heap with the n elements and repeatedly delete the maximum; the deleted maxima, written from the back of the same array, give sorted order, *in place*.

Algorithm 13: Heap-Sort(A)

```
1 Build-Max-Heap(A);
2 for last ← n to 2 do
3 |   swap A[1] ↔ A[last];
4 |   Max-Heapify(A, last − 1, 1);
```

Time $\mathcal{O}(n \log n)$ (each of the $n-1$ extractions costs $\mathcal{O}(\log n)$), space $\mathcal{O}(1)$ extra. Not stable.

Simple example: sorting 4, 1, 3, 2, 16, 9, 10, 14, 8, 7

Build-heap gives $16, 14, 10, 8, 7, 9, 3, 2, 4, 1$. Swap 16 with the last: $[1, 14, 10, 8, 7, 9, 3, 2, 4 \,|\, 16]$, heapify $\rightarrow [14, 8, 10, 4, 7, 9, 3, 2, 1 \,|\, 16]$, and so on until $[1, 2, 3, 4, 7, 8, 9, 10, 14, 16]$.

5.6 Building a heap

Method 1 (naive): n successive insertions $\Rightarrow \mathcal{O}(n \log n)$.
Method 2 (Floyd): given array $A[1..n]$, for $i = n$ down to 1: Max-Heapify(i). (Leaves are trivially heaps, so one starts at $i = \lfloor n/2 \rfloor$.) The naive analysis (n calls, each $\mathcal{O}(\log n)$) also says $\mathcal{O}(n \log n)$ – **but many operations are small, and the true cost is $\Theta(n)$.**

Algorithm 14: Build-Max-Heap(A)

```
1 for i ← ⌊n/2⌋ to 1 do
2 |   Max-Heapify(A, n, i)
```

Theorem

Build-Max-Heap takes $\Theta(n)$ time.

Proof via node heights

Heapify at a node of *height* h (height measured from the leaves) costs at most h swaps. An n-node heap has at most $\lceil n/2^{h+1} \rceil$ nodes of height h (about half of the nodes are leaves, a quarter have height 1, ...). Therefore

$$T(n) \le \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil h \;\le\; n \sum_{h=0}^{\infty} \frac{h}{2^h} \;=\; \mathcal{O}(n) \quad \left(\text{because } \sum_{h \ge 0} \frac{h}{2^h} = 2 \right).$$

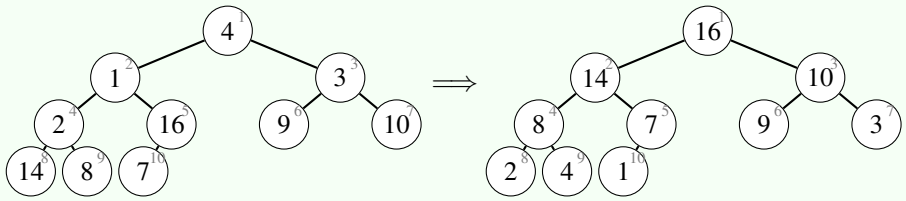
The lower bound $\Omega(n)$ is trivial: every element must be examined. □

Why $\sum_{h\geq 0} h/2^h = 2$

For $|x| < 1$, $\sum_{h\geq 0} x^h = \frac{1}{1-x}$. Differentiate: $\sum_{h\geq 1} hx^{h-1} = \frac{1}{(1-x)^2}$; multiply by x: $\sum hx^h = \frac{x}{(1-x)^2}$. At $x = \frac{1}{2}$ this is $\frac{1/2}{1/4} = 2$. □

Intuition (the swaps-counting view of the notes): a node at height 1 does at most 1 swap, there are $n/4$ of them; height 2: at most 2 swaps, $n/8$ nodes; .... Only the single root pays $\log n$. The sum $\frac{n}{4}\cdot 1 + \frac{n}{8}\cdot 2 + \frac{n}{16}\cdot 3 + \cdots$ is $\leq n$.

Simple example: Build-Max-Heap on 4, 1, 3, 2, 16, 9, 10, 14, 8, 7



$n = 10$, so start at $i = \lfloor 10/2 \rfloor = 5$. $i = 5$: $16 \geq 7$, nothing. $i = 4$: 2 vs children $14, 8$: swap with 14. $i = 3$: 3 vs $9, 10$: swap with 10. $i = 2$: 1 vs $14, 16$: swap with 16, then 1 vs child 7: swap. $i = 1$: 4 vs $16, 10$: swap with 16; then 4 vs $14, 7$: swap with 14; then 4 vs $2, 8$: swap with 8. Result $16, 14, 10, 8, 7, 9, 3, 2, 4, 1$ (verified by the program).

5.7 More priority-queue operations

- **Increase-Key**(i, v): set $A[i] = v$ (if larger) and sift up: $\mathcal{O}(\log n)$ (used by Dijkstra/Prim variants).
- **Min-heap:** reverse every comparison; DELETE-MIN then gives ascending order; the usual choice for Prim, Dijkstra, event simulation.
- **Applications:** operating-system schedulers, Huffman coding, k largest elements, merging sorted lists, median maintenance with two heaps.

5.8 C code

```c
/* 1-indexed array A[1..n]; A[0] unused.  MAX-heap. */
#define PARENT(i) ((i) / 2)
#define LEFT(i)   (2 * (i))
#define RIGHT(i)  (2 * (i) + 1)

static void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

/* Resolve a downward conflict at i: both subtrees of i are already max-heaps. O(log
   n) */
void max_heapify(int A[], int n, int i) {
    for (;;) {
        int l = LEFT(i), r = RIGHT(i), largest = i;
        if (l <= n && A[l] > A[largest]) largest = l;
        if (r <= n && A[r] > A[largest]) largest = r;
        if (largest == i) return;               /* heap property holds */
        swap(&A[i], &A[largest]);
        i = largest;                            /* keep sifting down   */
    }
}

/* Insert x (array must have room for n+1 elements). O(log n) */
```

```c
void heap_insert(int A[], int *n, int x) {
    A[++(*n)] = x;                              /* append at the end   */
    int j = *n;
    while (j > 1 && A[PARENT(j)] < A[j]) {      /* sift up             */
        swap(&A[PARENT(j)], &A[j]);
        j = PARENT(j);
    }
}

int heap_max(int A[]) { return A[1]; }          /* O(1) */

/* Remove and return the maximum. O(log n) */
int heap_delete_max(int A[], int *n) {
    int mx = A[1];
    A[1] = A[(*n)--];                           /* last element -> root */
    max_heapify(A, *n, 1);
    return mx;
}

/* Build a heap from an arbitrary array in Theta(n): sift down from the last internal
     node. */
void build_max_heap(int A[], int n) {
    for (int i = n / 2; i >= 1; i--)            /* leaves n/2+1..n are already heaps
    */
        max_heapify(A, n, i);
}

/* In-place heap sort: O(n log n) time, O(1) extra space. */
void heap_sort(int A[], int n) {
    build_max_heap(A, n);
    for (int last = n; last >= 2; last--) {
        swap(&A[1], &A[last]);                  /* max goes to its final place */
        max_heapify(A, last - 1, 1);            /* heap shrinks by one         */
    }
}

/* Increase the key at position i (priority-queue operation). O(log n) */
void heap_increase_key(int A[], int i, int newkey) {
    if (newkey < A[i]) return;
    A[i] = newkey;
    while (i > 1 && A[PARENT(i)] < A[i]) { swap(&A[PARENT(i)], &A[i]); i = PARENT(i);
     }
}
```

Time / space complexity

MAX	$\Theta(1)$
INSERT	$\mathcal{O}(\log n)$ (swaps $\le$ height $= \lfloor \log_2 n \rfloor$)
DELETE-MAX, MAX-HEAPIFY, INCREASE-KEY	$\mathcal{O}(\log n)$
BUILD-MAX-HEAP	$\Theta(n)$
Heap sort	$\Theta(n \log n)$ time, $\mathcal{O}(1)$ extra space
Space	$\Theta(n)$, no pointers

Chapter 6

Graphs

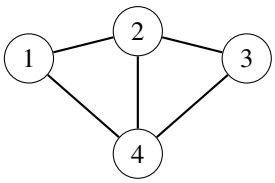
6.1 Basic definitions

Graph

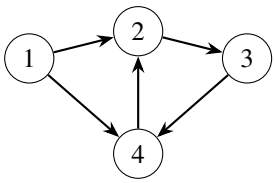
A graph $G = (V, E)$ consists of a set of **vertices** (nodes) V and a set of **edges** E. In an **undirected** graph an edge is an unordered pair $\{u, v\}$; in a **directed** graph (digraph) it is an ordered pair (u, v) (an arrow $u \to v$). We write $n = |V|$, $m = |E|$.

- **Degree** of v: number of incident edges (digraphs: in-degree and out-degree). **Handshake lemma:** $\sum_v \deg(v) = 2m$ (each edge is counted at both ends).
- **Path:** sequence of vertices with consecutive ones joined by edges. **Cycle:** path that returns to its start. A graph is **connected** if there is a path between every pair of vertices.
- **Complete graph** K_n: every pair adjacent, $m = \binom{n}{2} = n(n-1)/2$ edges. **Path** P_n, **cycle** C_n and **star** (one centre joined to all others) are other standard families. A **tree** is a connected graph without cycles: $m = n - 1$.
- **Sparse** graph: $m = \mathcal{O}(n)$; **dense**: $m = \Theta(n^2)$. Since $m \le n(n-1)/2$ in any simple undirected graph, $m = \mathcal{O}(n^2)$.

Why graphs? Road maps, social networks (people = vertices, friendships = edges), the web (pages and links), task dependencies, circuits, state machines, finding connected groups, shortest routes, schedules.



undirected

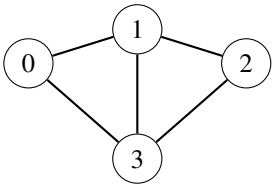


directed

6.2 Representations

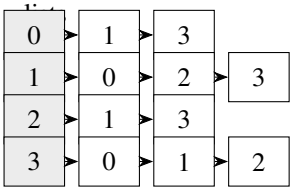
Let the vertices be $0, \ldots, n-1$.

- **Adjacency matrix** $A[n][n]$, $A[u][v] = 1$ iff $(u, v) \in E$ (symmetric if undirected).
- **Adjacency list:** an array of n linked lists; the list of u holds the neighbours of u.



matrix

0	1	0	1
1	0	1	1
0	1	0	1
1	1	1	0



	Adjacency matrix	Adjacency list
Space	$\Theta(n^2)$	$\Theta(n+m)$
Is (u,v) an edge?	$\mathcal{O}(1)$	$\mathcal{O}(\deg u)$
List all neighbours of u	$\Theta(n)$	$\Theta(\deg u)$
Visit every edge	$\Theta(n^2)$	$\Theta(n+m)$
Best for	dense graphs, fast edge test	sparse graphs (almost always)

```c
/* ---------- adjacency list ---------- */
typedef struct Edge { int to; struct Edge *next; } Edge;
typedef struct { int n; Edge *adj[MAXV]; } Graph;

void g_init(Graph *g, int n) { g->n = n; for (int i = 0; i < n; i++) g->adj[i] = NULL; }
void g_add_directed(Graph *g, int u, int v) {          /* keeps insertion order */
    Edge *e = malloc(sizeof(Edge)); e->to = v; e->next = NULL;
    Edge **p = &g->adj[u]; while (*p) p = &(*p)->next; *p = e;
}
void g_add_undirected(Graph *g, int u, int v) { g_add_directed(g, u, v);
    g_add_directed(g, v, u); }
void g_free(Graph *g) {
    for (int i = 0; i < g->n; i++) for (Edge *e = g->adj[i]; e;) { Edge *t = e->next;
     free(e); e = t; }
}
/* ---------- adjacency matrix (for comparison) ---------- */
typedef struct { int n; bool a[MAXV][MAXV]; } MGraph;      /* O(n^2) space, O(1) edge
     test */
```

6.3 Exploring a graph: depth-first search

Question 1: which vertices can be reached from a vertex s? **Idea:** like exploring a maze with chalk and string: walk along unexplored edges, mark vertices you visit, and backtrack when stuck.

Algorithm 15: Explore(w) (DFS from w)

```
1 visited(w) ← true;
2 foreach edge (w,z) ∈ E do
3 |   if visited(z) = false then Explore(z);
```

Correctness

Explore(s) visits exactly the vertices reachable from s.

Proof

(*Only reachable ones.*) A vertex is first visited through an edge from an already visited vertex, so it is reachable. (*All reachable ones.*) Suppose u is reachable but never visited; on a path $s = p_0, p_1, \ldots, p_r = u$ let p_j be the first unvisited vertex ($j \geq 1$). Then p_{j-1} was visited, and when Explore(p_{j-1}) scanned its edge to p_j, either p_j was already visited or it was explored. Contradiction. □

Running time. Each vertex is explored once and each adjacency list is scanned once: $\mathcal{O}(n+m)$ (with a matrix: $\mathcal{O}(n^2)$). To cover the *whole* graph, restart from every unvisited vertex:

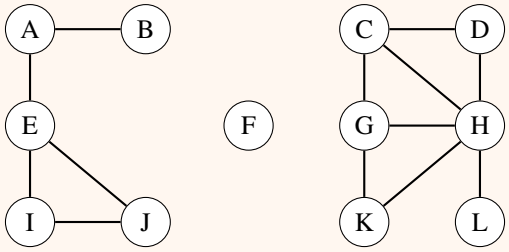
Algorithm 16: DFS(G)

```
1 foreach v ∈ V do visited(v) ← false;
2 foreach v ∈ V do
3 |   if visited(v) = false then Explore(v);
```

6.3.1 Connected components

Maintain a counter cc; each time DFS starts a new tree, increment cc, and in the *previsit* of every vertex set $ccnum(w) \leftarrow cc$. Two vertices are in the same connected component iff they have the same $ccnum$.

From your notes: 12 vertices, 3 components



vertex	A	B	C	D	E	F	G	H	I	J	K	L
ccnum	1	1	2	2	1	3	2	2	1	1	2	2

Starting DFS at A finds $\{A,B,E,I,J\}$; the next unvisited vertex in alphabetical order, C, finds $\{C,D,G,H,K,L\}$; F is alone. Cost $\mathcal{O}(n+m)$ in total.

```c
/* ---------- DFS with clock, pre/post numbers, component ids ---------- */
static bool visited[MAXV];
static int pre[MAXV], post[MAXV], parent[MAXV], clock_, ccnum[MAXV], cc;
static int order[MAXV], ordn;                                  /* vertices by finishing
    time */

static void explore(const Graph *g, int w, int p) {
    visited[w] = true; parent[w] = p;
    ccnum[w] = cc;                                             /* previsit: record
    component */
    pre[w] = clock_++;                                         /* previsit  */
    for (Edge *e = g->adj[w]; e; e = e->next)
        if (!visited[e->to]) explore(g, e->to, w);
    post[w] = clock_++;                                        /* postvisit */
    order[ordn++] = w;
}
/* DFS over all vertices, optionally in a given vertex order (NULL = 0..n-1) */
void dfs(const Graph *g, const int *vorder) {
    for (int v = 0; v < g->n; v++) visited[v] = false;
    clock_ = 1; cc = 0; ordn = 0;
    for (int i = 0; i < g->n; i++) {
        int v = vorder ? vorder[i] : i;
        if (!visited[v]) { cc++; explore(g, v, -1); }          /* new component / new
     tree */
    }
}
```

6.4 Pre- and post-visit numbers

Keep a **clock** starting at 1; *previsit*(w): $pre(w) \leftarrow clock$; $clock{+}{+}$; *postvisit*(w) (after all edges of w are explored): $post(w) \leftarrow clock$; $clock{+}{+}$. Vertex w is "active" during the interval $[pre(w), post(w)]$.

Algorithm 17: Explore(w) with clock

```
1 visited(w) ← true; pre(w) ← clock; clock ← clock + 1;
2 foreach edge (w, z) ∈ E do
3 |   if visited(z) = false then Explore(z);
4 post(w) ← clock; clock ← clock + 1;
```

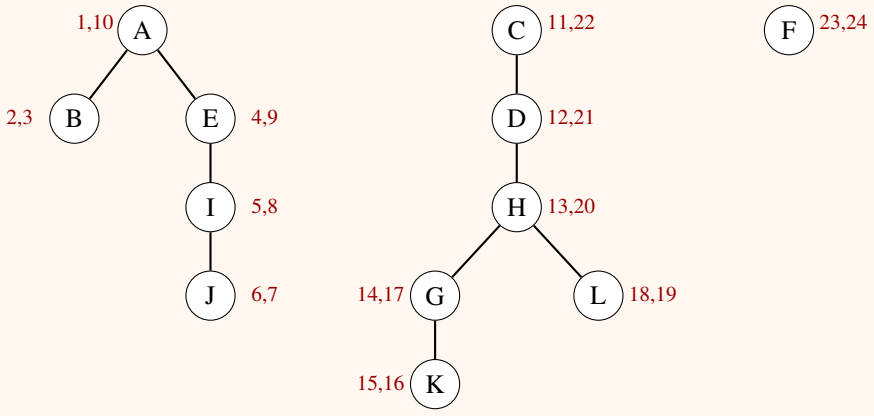
Parenthesis property

For any two vertices u, v the intervals $[pre(u), post(u)]$ and $[pre(v), post(v)]$ are either **disjoint** or **one contains the other**; never partially overlapping. Containment $\Leftrightarrow$ the inner vertex is a **descendant** of the outer in the DFS forest.

Proof

Let $pre(u) < pre(v)$. If v is discovered while u is still active ($pre(v) < post(u)$), then Explore(v) is a nested recursive call, so it *finishes before* Explore(u) does: $[pre(v), post(v)] \subset [pre(u), post(u)]$. Otherwise v is discovered after u finished and the intervals are disjoint. □

From your notes: DFS forest of the undirected example, $[pre, post]$



Intervals nest along tree paths: $[4,9] \supset [5,8] \supset [6,7]$; vertices in different trees (A and C) have disjoint intervals $[1,10]$, $[11,22]$. Our program prints exactly these numbers.

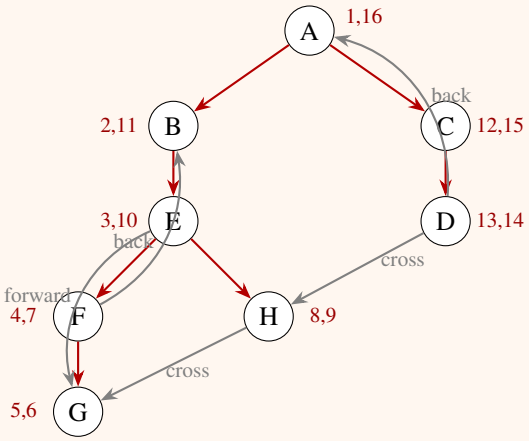
6.5 DFS in directed graphs

In an undirected graph DFS produces *tree edges* and *back edges*. In a digraph, for an edge $u \to v$ there are four types:

Type	Meaning	Intervals
Tree edge	edge used by DFS to discover v	$[v]$ inside $[u]$
Forward edge	u to a non-child descendant v	$[v]$ inside $[u]$
Back edge	descendant u to ancestor v	$[u]$ inside $[v]$
Cross edge	to neither descendant nor ancestor	$[v]$ entirely *before* $[u]$

A cross edge always goes from a later to an earlier finished vertex: $post(v) < pre(u)$. (A cross edge from u to a vertex discovered *later* cannot exist, since v would have been explored from u.) Code: `edge_type`.

From your notes: DFS tree with the other edges



Red = tree edges. $F \to B$ and $D \to A$ point from descendant to ancestor (**back**); $E \to G$ jumps from E $[3,10]$ to a grandchild G $[5,6]$ (**forward**); $H \to G$ and $D \to H$ go to vertices finished earlier (G: $[5,6]$ before H's 8; H: $[8,9]$ before D's 13) (**cross**). The program classifies all 12 edges identically.

```
/* classify edge u->v after a DFS (directed graphs) */
const char *edge_type(int u, int v) {
    if (pre[u] < pre[v] && post[v] < post[u])              /* v is a descendant of u
    */
        return parent[v] == u ? "tree" : "forward";
    if (pre[v] < pre[u] && post[u] < post[v]) return "back";
    return "cross";                                        /* post[v] < pre[u] */
}
```

6.6 DAGs and topological sort

A **DAG** is a directed acyclic graph. Typical use: tasks with prerequisites (courses, build systems). A **topological order** lists the vertices so that every edge goes from earlier to later. It exists iff the graph is a DAG.

Theorem

A directed graph has a cycle $\iff$ DFS finds a **back edge**.

Proof

($\Leftarrow$) A back edge $u \to v$ (v an ancestor of u) together with the tree path $v \rightsquigarrow u$ forms a cycle.
($\Rightarrow$) Let a cycle exist and let v be the first of its vertices discovered. All other cycle vertices are reachable from v via unvisited vertices, so they become descendants of v. The cycle edge $u \to v$ entering v then goes from a descendant to an ancestor: a back edge. □

Key lemma

In a DAG, for every edge $u \to v$ we have $post(u) > post(v)$.

Proof

By the previous theorem there is no back edge. A tree/forward edge has $[v]$ inside $[u]$, so $post(v) < post(u)$. A cross edge has $post(v) < pre(u) < post(u)$. □

Algorithm: run DFS and output vertices in **decreasing post-visit number**, $\mathcal{O}(n+m)$. (Equivalently: repeatedly remove a *source* – a vertex without incoming edges – which exists in every DAG: walk backwards along edges; without cycles the walk must stop.)

Simple example: tasks A→B, A→C, B→D, C→D, D→E

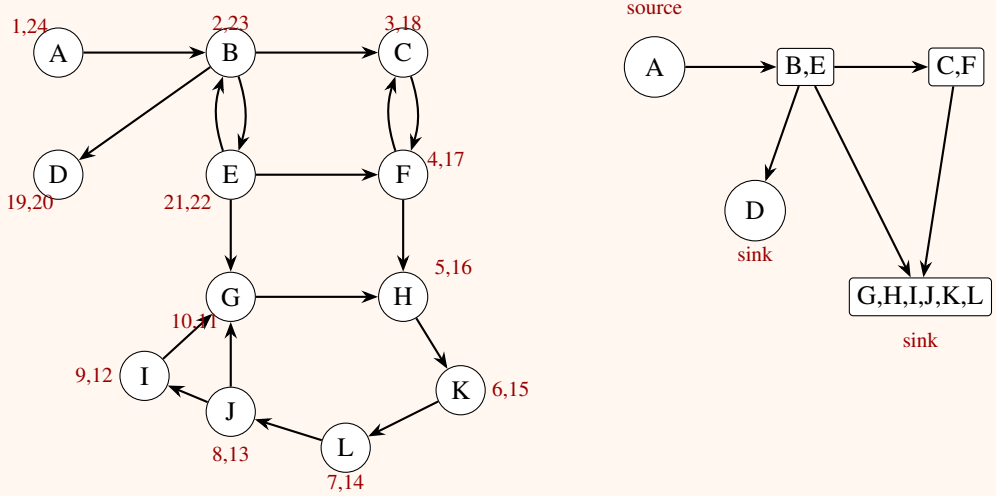
DFS from A (adjacency order): $A(1) \to B(2) \to D(3) \to E(4,5)$, then D finishes (6), B finishes (7); back at A we explore C(8,9) (its edge to D is already visited), and A finishes (10). Intervals: $A[1,10]$, $B[2,7]$, $D[3,6]$, $E[4,5]$, $C[8,9]$. Decreasing post: $A(10), C(9), B(7), D(6), E(5)$, i.e. **A, C, B, D, E**; every edge points right. (This is what the program prints.)

```c
/* topological sort of a DAG: vertices in decreasing post number */
void topo_sort(const Graph *g, int out[]) {
    dfs(g, NULL);
    for (int i = 0; i < g->n; i++) out[i] = order[g->n - 1 - i];  /* order[] is
    increasing post */
}
```

6.7 Strongly connected components

In a digraph, u and v are **strongly connected** if u reaches v *and* v reaches u. This is an equivalence relation; its classes are the **strongly connected components** (SCCs). **Meta-graph:** contract each SCC to one vertex. **The meta-graph is a DAG**: a cycle of SCCs would merge them into one (every vertex could reach every other). So SCCs have **source** components (no incoming edges) and **sink** components (no outgoing edges).

From your notes: digraph with 12 vertices, $[pre, post]$ and meta-graph



The meta-graph on the right is a DAG with source $\{A\}$ and sinks $\{D\}$ and $\{G,H,I,J,K,L\}$. Notice the *highest* post number (24, vertex A) lies in a *source* component.

Finding the SCCs (Kosaraju–Sharir; the 3-step algorithm in the notes)

1. Run DFS on the **reverse graph** G^R (all edges flipped); record post numbers.
2. Order the vertices by **decreasing** post number from step 1.
3. Run DFS (the connected-components loop) on the **original** graph G, starting new trees in that order. Each DFS tree is one SCC.

Lemma

If $C \to C'$ is an edge between different SCCs, then $\max_{v \in C} post(v) > \max_{v \in C'} post(v)$.

Proof

Let v be the first vertex discovered in $C \cup C'$. *Case $v \in C$:* everything in $C \cup C'$ is reachable from v, so all become descendants of v and $post(v)$ is the largest of both sets. *Case $v \in C'$:* C' cannot reach C (the meta-graph is a DAG), so the exploration of C' finishes before any vertex of C is discovered, so every post number in C' is smaller than every one in C. □

Why the algorithm works

G and G^R have the same SCCs, with the meta-edges reversed. Applying the lemma to G^R: the vertex with the largest post number in step 1 lies in a *source* SCC of G^R, i.e. a *sink* SCC of G. In step 3, a DFS from a vertex of a sink SCC of G cannot leave it, so it visits *exactly* that SCC. Remove it; the next-highest unvisited vertex lies in a sink SCC of what remains, and so on (induction). Two DFS runs: $\mathcal{O}(n+m)$. □

From your notes: result on the example

The program prints the five SCCs: $\{G,H,I,J,K,L\}$ (found first: the sink), $\{D\}$, $\{C,F\}$, $\{B,E\}$, $\{A\}$.

```
/* strongly connected components: DFS on G^R, then DFS on G in decreasing post order */
int scc(const Graph *g, int comp[]) {
    Graph r; g_init(&r, g->n);
    for (int u = 0; u < g->n; u++) for (Edge *e = g->adj[u]; e; e = e->next)
    g_add_directed(&r, e->to, u);
    dfs(&r, NULL);                                          /* step 1: post numbers of G^R */
    int ord[MAXV]; for (int i = 0; i < g->n; i++) ord[i] = order[g->n - 1 - i];  /* step 2 */
    g_free(&r);
    dfs(g, ord);                                            /* step 3: components of G in that order */
    for (int v = 0; v < g->n; v++) comp[v] = ccnum[v];
    return cc;
}
```

6.8 Breadth-first search (supplement)

DFS goes deep; **BFS** explores in layers by distance, using a queue. It computes **shortest-path lengths (number of edges)** from a source in unweighted graphs.

Algorithm 18: BFS(G,s)

```
dist(v) ← ∞ for all v; dist(s) ← 0; enqueue s;
while queue not empty do
    u ← dequeue;
    foreach edge (u,z) do
        if dist(z) = ∞ then
            dist(z) ← dist(u) + 1; enqueue z
```

Time $\mathcal{O}(n+m)$: each vertex is enqueued once and each adjacency list scanned once. **Correctness sketch:** vertices leave the queue in non-decreasing order of $dist$; a vertex at true distance d is discovered when its neighbour at distance $d-1$ is dequeued, so $dist(v)$ equals the true distance (induction on d). On the 12-vertex undirected graph from C the program gives $D,G,H = 1$ and $K,L = 2$; vertices in other components stay unreachable.

```c
/* BFS: shortest path lengths (edges) from s */
void bfs(const Graph *g, int s, int dist[]) {
    int q[MAXV], head = 0, tail = 0;
    for (int v = 0; v < g->n; v++) dist[v] = -1;
    dist[s] = 0; q[tail++] = s;
    while (head < tail) {
        int u = q[head++];
        for (Edge *e = g->adj[u]; e; e = e->next)
            if (dist[e->to] < 0) { dist[e->to] = dist[u] + 1; q[tail++] = e->to; }
    }
}
```

	DFS	BFS
Data structure	recursion / stack	queue
Finds	components, cycles, topological order, SCCs	shortest paths (unweighted)
Time	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$

6.9 Planar graphs and Euler's formula

A graph is **planar** if it can be drawn in the plane without crossing edges. The drawing divides the plane into faces (including the outer one).

Euler's formula

For a connected planar graph with n vertices, m edges and f faces: $n-m+f=2$.

Idea

A tree has $m=n-1$ and one face: $n-(n-1)+1=2$. Adding an edge to a connected planar graph either closes a cycle (splitting one face into two: m and f both rise by 1) or attaches a new vertex (both n and m rise by 1); the quantity $n-m+f$ never changes. □

Consequences. Every face is bounded by at least 3 edges and every edge borders at most 2 faces, so $3f \le 2m$. With Euler: $m \le 3n-6$. Hence $\sum \deg = 2m \le 6n-12$, the average degree is below 6, so **every planar graph has a vertex of degree** ≤ 5 (in particular ≤ 6, as in the notes). Thus planar graphs are sparse: $m=\mathcal{O}(n)$, and K_5 ($m=10>3\cdot 5-6=9$) is not planar.

Time / space complexity

Explore / DFS / BFS (adjacency lists)	$\Theta(n+m)$
Connected components, topological sort	$\Theta(n+m)$
Strongly connected components	$\Theta(n+m)$ (two DFS)
Adjacency matrix versions	$\Theta(n^2)$

Chapter 7

Minimum Spanning Trees

7.1 Spanning trees and the MST problem

Definitions

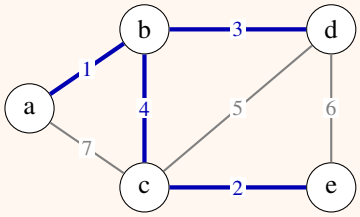
Let $G = (V, E)$ be a connected undirected graph with edge weights $w : E \to \mathbb{R}$. A **spanning tree** is a subset $T \subseteq E$ that connects all vertices and has no cycle (equivalently: a tree on V, so $|T| = |V| - 1$). A **minimum spanning tree (MST)** is a spanning tree of minimum total weight $w(T) = \sum_{e \in T} w(e)$.

Motivation: connect n sites (cities, houses, network switches) with the least total cable. Facts used throughout:

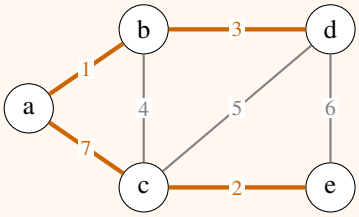
- A tree on n vertices has exactly $n - 1$ edges; adding any extra edge creates exactly one cycle; removing a tree edge disconnects the tree into two components.
- If all weights are distinct the MST is **unique**; with ties several MSTs may exist (all have the same weight).
- A graph may have exponentially many spanning trees (n^{n-2} for K_n, Cayley), so brute force is hopeless.

From your notes: five vertices, MST of weight 10

Edges: ab=1, ac=7, bc=4, bd=3, cd=5, ce=2, de=6.



MST: $1 + 3 + 4 + 2 = 10$



another spanning tree: 13

7.2 The generic algorithm and the cut property

Grow a set A of edges, always keeping $A \subseteq$ some MST.

Algorithm 19: Generic-MST(G, w)

```
A ← ∅;
while A does not form a spanning tree do
    find an edge (u, v) that is safe for A;
    A ← A ∪ {(u, v)}
return A;
```

An edge $(u, v) \notin A$ is **safe** for A if some MST contains $A \cup \{(u, v)\}$.

Cut

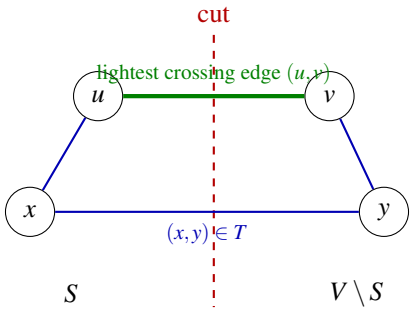
A **cut** is a partition $(S, V \setminus S)$ of the vertices. An edge **crosses** the cut if its endpoints are on different sides. A cut **respects** A if no edge of A crosses it.

Cut property

Let A be contained in some MST, $(S, V \setminus S)$ a cut that respects A, and (u,v) a minimum-weight edge crossing the cut. Then (u,v) is safe for A.

Exchange argument

Let T be an MST with $A \subseteq T$. If $(u,v) \in T$ we are done. Otherwise $T \cup \{(u,v)\}$ contains a unique cycle p through u and v. Since $u \in S$ and $v \notin S$ (or vice versa), the cycle must cross the cut at least once more, at some edge $(x,y) \in T$, $(x,y) \neq (u,v)$. Because the cut respects A, $(x,y) \notin A$. Now (u,v) is lightest across the cut, so $w(u,v) \leq w(x,y)$. Define $T' = T \setminus \{(x,y)\} \cup \{(u,v)\}$. It is still a spanning tree (we broke the only cycle) and $w(T') = w(T) - w(x,y) + w(u,v) \leq w(T)$. So T' is also an MST, and it contains $A \cup \{(u,v)\}$. □



Replace the tree edge (x,y) by the lighter (or equal) (u,v): the weight does not increase.

7.3 Kruskal's algorithm

Idea: consider the edges in increasing order of weight; **accept an edge if it joins two different components** of the forest built so far (so it creates no cycle), otherwise reject it. Initially every vertex is its own component.

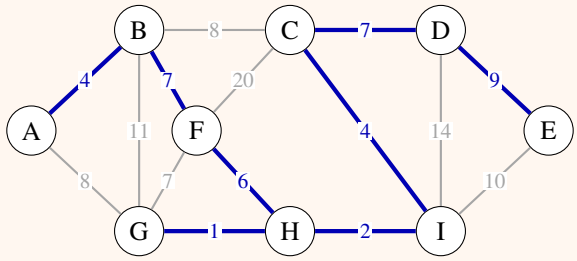
Algorithm 20: Kruskal(G,w)

```
A ← ∅;
foreach v ∈ V do
    MakeSet(v)
sort E by weight into non-decreasing order;
foreach (u,v) ∈ E in this order do
    if FindSet(u) ≠ FindSet(v) then
        A ← A ∪ {(u,v)}; Union(u,v)
return A;
```

From your notes: the 9-vertex example from the slides

Label the vertices A (far left), B (top-left), C (top-middle), D (top-right), E (far right), F (centre), G (bottom-left), H (bottom-middle), I (bottom-right). Edges: AB4, AG8, BC8, BG11, BF7, CF20, CD7, CI4, DI14, DE9, EI10, FG7, FH6, GH1, HI2. Sorted processing:

Weight	Edge	Components joined?	Decision
1	GH	$\{G\},\{H\}$	**accept**
2	HI	$\{G,H\},\{I\}$	**accept**
4	AB	$\{A\},\{B\}$	**accept**
4	CI	$\{C\},\{G,H,I\}$	**accept**
6	FH	$\{F\},\{G,H,I,C\}$	**accept**
7	BF	$\{A,B\},\{F,G,H,I,C\}$	**accept**
7	CD	$\{D\},\ldots$	**accept**
7	FG	same component	reject (cycle)
8	AG, BC	same component	reject
9	DE	$\{E\},\ldots$	**accept** (8 edges: done)



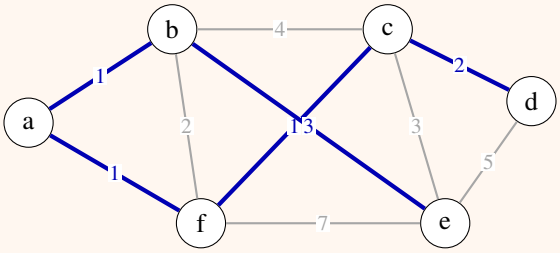
MST weight $= 1+2+4+4+6+7+7+9 =$ **40** (both Kruskal and Prim in our program print 40).

Simple example: triangle with a tail

Vertices a,b,c,d; edges ab=1, bc=2, ac=3, cd=5. Sorted: ab accept, bc accept, ac rejected (a,c already connected through b), cd accept. MST $= 1+2+5 = 8$.

From your notes: six vertices, MST weight 8

Edges: ab=1, af=1, bc=4, bf=2, be=3, cf=1, cd=2, ce=3, de=5, ef=7. Kruskal: ab ✓, af ✓, cf ✓; bf rejected (cycle a-b-f); cd ✓; be ✓; ce, bc, de, ef all rejected. Weight $1+1+1+2+3 = 8$.



7.3.1 Correctness

Kruskal returns an MST

Invariant: the set A is contained in some MST. Initially $A = \emptyset$. Suppose Kruskal accepts (u,v), joining two different components; let C_1 be the component of u in the forest (V,A). Consider the cut $(C_1, V \setminus C_1)$. It *respects* A (components are closed under A-edges). The edge (u,v) crosses it, and it is a lightest crossing edge: any lighter edge crossing the cut would have been examined earlier, and, since its endpoints lie in different components then as well (components only grow), it would have been accepted, contradicting that it still crosses. By the cut property (u,v) is safe, so the invariant holds. The final A is a spanning tree contained in an MST, hence an MST. □

7.3.2 Running time

Sorting costs $\mathcal{O}(E \log E) = \mathcal{O}(E \log V)$ (as $E \le V^2$). The loop makes $2E$ FindSet calls and at most $V-1$ Union calls. With a good disjoint-set structure (next section) these cost $\mathcal{O}(E + V \log V)$ or less, so **Kruskal runs in** $\mathcal{O}(E \log V)$. A naive implementation (e.g. relabelling a component array on every union: $\mathcal{O}(V)$ per union) would cost $\mathcal{O}(E \log E + V^2)$.

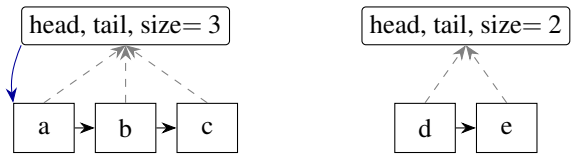
7.4 Disjoint-set (union-find) data structure

Maintains a collection of disjoint sets, each with a **representative**:

- MAKESET(x): new set $\{x\}$. FINDSET(x): representative of the set containing x (two elements are in the same set iff their representatives are equal). UNION(x, y): merge the sets containing x and y.

7.4.1 Linked lists with weighted union

Each set is a linked list whose first element is the representative; every element points to a **metadata node** holding the head, the tail and the *size* of its set.



FINDSET follows the metadata pointer to the head: $\mathcal{O}(1)$. UNION: append the **smaller** list to the larger one, update the smaller list's elements to point to the larger metadata (cost $\propto$ size of the smaller set).

Weighted union

With union by size, any sequence of m operations of which n are MAKESET takes $\mathcal{O}(m + n \log n)$ time.

Proof

Fix an element x. Each time its metadata pointer is rewritten, x was in the *smaller* (or equal) set, so the size of x's set at least **doubles**. A set never exceeds n elements, hence x's pointer changes at most $\log_2 n$ times. Summing over the n elements: at most $n \log_2 n$ pointer updates in total; every operation also pays $\mathcal{O}(1)$. □

(The program merges 4096 singletons pairwise and counts exactly $24576 = \frac{1}{2} n \log_2 n$ updates.) With this, Kruskal's disjoint-set work is $\mathcal{O}(E + V \log V)$.

```c
/* ============ Disjoint sets, version 1: linked lists + weighted union ============ */
typedef struct Meta Meta;
typedef struct LNode { int val; struct LNode *next; Meta *meta; } LNode;
struct Meta { LNode *head, *tail; int size; };            /* head = representative */

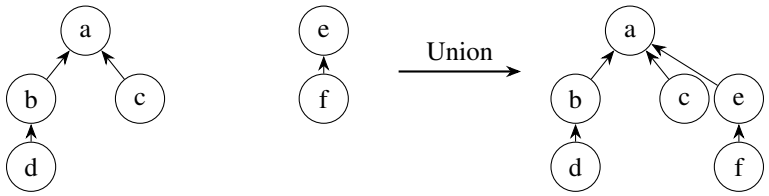
long meta_updates = 0;                                    /* counts pointer rewrites */

LNode *ll_makeset(int x) {                                /* O(1) */
    LNode *n = malloc(sizeof(LNode)); Meta *m = malloc(sizeof(Meta));
    n->val = x; n->next = NULL; n->meta = m;
    m->head = m->tail = n; m->size = 1;
    return n;
}
```

```
LNode *ll_findset(LNode *x) { return x->meta->head; }      /* O(1) */
void ll_union(LNode *x, LNode *y) {                        /* append the SMALLER list
    to the larger */
    Meta *S = x->meta, *T = y->meta;
    if (S == T) return;
    if (S->size > T->size) { Meta *t = S; S = T; T = t; } /* now |S| <= |T| */
    T->tail->next = S->head;                               /* append S after T */
    T->tail = S->tail;
    T->size += S->size;
    for (LNode *p = S->head; p; p = p->next) { p->meta = T; meta_updates++; }
    free(S);
}
```

7.4.2 Disjoint-set forests (supplement)

Represent each set as a rooted tree with parent pointers; the root is the representative. FIND follows parents to the root; UNION makes one root the child of the other.



Two heuristics make this nearly constant time: **union by rank** (attach the shallower tree below the deeper one, keeping height $\mathcal{O}(\log n)$) and **path compression** (during FIND, point every visited node directly at the root). Together a sequence of m operations costs $\mathcal{O}(m\,\alpha(n))$, where α is the inverse Ackermann function ($\alpha(n) \le 4$ for every practical n), so Kruskal's cost is dominated by sorting.

```
/* ============ Disjoint sets, version 2: forest + union by rank + path compression
    ============ */
#define MAXV 64
int parent[MAXV], rnk[MAXV];

void makeset(int x) { parent[x] = x; rnk[x] = 0; }         /* root is its own parent
    */
int find(int x) {                                          /* with path compression
    */
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}
int unite(int a, int b) {                                  /* returns 0 if already
    together */
    int ra = find(a), rb = find(b);
    if (ra == rb) return 0;
    if (rnk[ra] < rnk[rb]) { int t = ra; ra = rb; rb = t; }
    parent[rb] = ra;                                       /* shorter tree under
    taller */
    if (rnk[ra] == rnk[rb]) rnk[ra]++;
    return 1;
}
```

```
/* ============ Kruskal ============ */
typedef struct { int u, v, w; } WEdge;
static int cmp(const void *a, const void *b) { return ((const WEdge *)a)->w - ((const
     WEdge *)b)->w; }

/* edges[0..m-1], vertices 0..n-1; MST edges are written to out[]; returns total
    weight */
int kruskal(int n, WEdge edges[], int m, WEdge out[], int *cnt) {
```

```
    int total = 0; *cnt = 0;
    for (int v = 0; v < n; v++) makeset(v);
    qsort(edges, m, sizeof(WEdge), cmp);                    /* O(E log E) */
    for (int i = 0; i < m && *cnt < n - 1; i++)
        if (unite(edges[i].u, edges[i].v)) {                /* Find(u) != Find(v) */
            out[(*cnt)++] = edges[i];
            total += edges[i].w;
        }
    return total;
}
```

7.5 Prim's algorithm (supplement)

Idea: grow *one* tree from a start vertex; at each step add the lightest edge connecting the tree to a vertex outside it. Kruskal grows a forest and merges components; Prim grows a single component.

Algorithm 21: Prim(G, w, s)

```
foreach v ∈ V do
    key(v) ← ∞; parent(v) ← NIL
key(s) ← 0; Q ← V (min-priority queue keyed by key);
while Q ≠ ∅ do
    u ← Extract-Min(Q);
    foreach neighbour v of u with v ∈ Q and w(u,v) < key(v) do
        parent(v) ← u; key(v) ← w(u,v)
```

Correctness: with $S =$ the vertices already in the tree, the cut $(S, V \setminus S)$ respects the tree edges and the chosen edge is the lightest crossing it, so it is safe by the cut property.

From your notes: Prim on the 9-vertex example, start at A

Order of additions: $AB(4)$, $BF(7)$, $FH(6)$, $HG(1)$, $HI(2)$, $IC(4)$, $CD(7)$, $DE(9)$: total 40, the same weight as Kruskal.

```
/* ============ Prim (array version, O(V^2)); W[u][v] = 0 means no edge ============ */
int prim(int n, int W[][MAXV], int start, int par[]) {
    int key[MAXV], in[MAXV] = {0}, total = 0;
    for (int v = 0; v < n; v++) { key[v] = INT_MAX; par[v] = -1; }
    key[start] = 0;
    for (int it = 0; it < n; it++) {
        int u = -1;
        for (int v = 0; v < n; v++) if (!in[v] && (u < 0 || key[v] < key[u])) u = v;
        in[u] = 1; total += key[u];                         /* cheapest edge leaving the tree */
        for (int v = 0; v < n; v++)
            if (W[u][v] && !in[v] && W[u][v] < key[v]) { key[v] = W[u][v]; par[v] = u; }
    }
    return total;
}
```

Time: array version $\mathcal{O}(V^2)$ (good for dense graphs); with a binary heap $\mathcal{O}(E \log V)$; with a Fibonacci heap $\mathcal{O}(E + V \log V)$.

Time / space complexity

Kruskal	$\mathcal{O}(E \log E) = \mathcal{O}(E \log V)$; sorting dominates
Prim (array / binary heap)	$\mathcal{O}(V^2)$ / $\mathcal{O}(E \log V)$
Union-find (weighted lists)	$\mathcal{O}(m + n \log n)$; forests with both heuristics $\mathcal{O}(m\,\alpha(n))$
Choice	sparse graph: Kruskal or heap-Prim; dense graph: array-Prim

Chapter 8

Quick Reference

8.1 Complexity cheat sheet

Structure / algorithm	Operation	Time	Remark
AVL tree	search / insert / delete	$\mathcal{O}(\log n)$	height $\leq 1.44\log_2 n$; insert ≤ 1 rotation event
B-tree (min degree t)	search / insert / delete	$\mathcal{O}(\log_t n)$ disk reads	height $\leq \log_t \frac{n+1}{2}$
B+ tree	search / range	$\mathcal{O}(\log n)$ + output pages	leaves linked
Hash table (chaining)	search / insert / delete	$\Theta(1+\alpha)$ avg.	worst case $\Theta(n)$
Hash table (open addr.)	unsuccessful search	$\leq 1/(1-\alpha)$	$\alpha < 1$
Universal hashing	collisions with a key	$\leq (N-1)/M$ expected	any input
Perfect hashing	search	$\mathcal{O}(1)$ worst case	space $< 2n$ expected
Bloom filter	insert / query	$\mathcal{O}(k)$	$p \approx (1-e^{-kn/m})^k$, no false negatives
Binary heap	insert / delete-max	$\mathcal{O}(\log n)$	max in $\mathcal{O}(1)$
Build heap		$\Theta(n)$	Heap sort $\Theta(n\log n)$
DFS / BFS		$\Theta(n+m)$	adjacency lists
Topological sort, SCC		$\Theta(n+m)$	decreasing post; two DFS
Kruskal		$\mathcal{O}(m\log n)$	sort + union-find
Prim		$\mathcal{O}(m\log n)$ or $\mathcal{O}(n^2)$	heap or array

8.2 Key formulas

AVL minimum nodes	$N(h) = 1+N(h-1)+N(h-2)$, $N(h)+1 = F_{h+3}$, $h \leq 1.44\log_2(n+1)$
B-tree height	$h \leq \log_t \frac{n+1}{2}$; node has $t-1 \leq n \leq 2t-1$ keys
Heap indices (1-based)	parent $\lfloor i/2 \rfloor$, children $2i, 2i+1$; height $\lfloor \log_2 n \rfloor$
Load factor	$\alpha = n/m$; empty slots $\approx e^{-\alpha}$
Open addressing	unsuccessful $\leq \frac{1}{1-\alpha}$, successful $\leq \frac{1}{\alpha}\ln\frac{1}{1-\alpha}$
Bloom filter	$k_{opt} = \frac{m}{n}\ln 2$, $m = -\frac{n\ln p}{(\ln 2)^2}$
Handshake	$\sum \deg = 2m$
Planar graph	$n-m+f = 2$, $m \leq 3n-6$
Tree / spanning tree	$n-1$ edges

8.3 When to use what

- Ordered data in memory with worst-case guarantees → **AVL tree**. Data on disk / in blocks → **B-tree**, and with range scans → **B+ tree**.
- Exact membership / dictionary, no order needed → **hash table**. Approximate membership in tiny space → **Bloom filter**.
- Repeatedly need the largest (smallest) item → **heap**; sorting in place → heap sort.
- Reachability, components, cycles, topological order, SCC → **DFS**; fewest edges → BFS.
- Cheapest way to connect everything → **MST** (Kruskal for sparse graphs, Prim for dense).